

ANN: Definition

The definition of ANN is given in 3.1 points.
Indeed, an ANN is a machine that is completely specified once we define its:

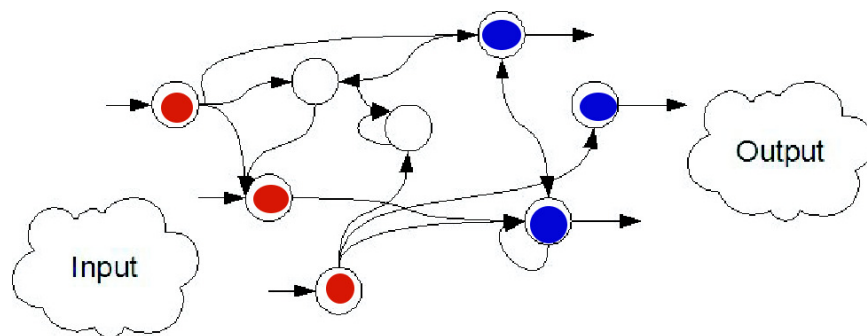
1. architecture
2. dynamics
3. learning

3.1 generalization

In the following we will give a definition of each one of these quantities.

ANN definition #1: architecture

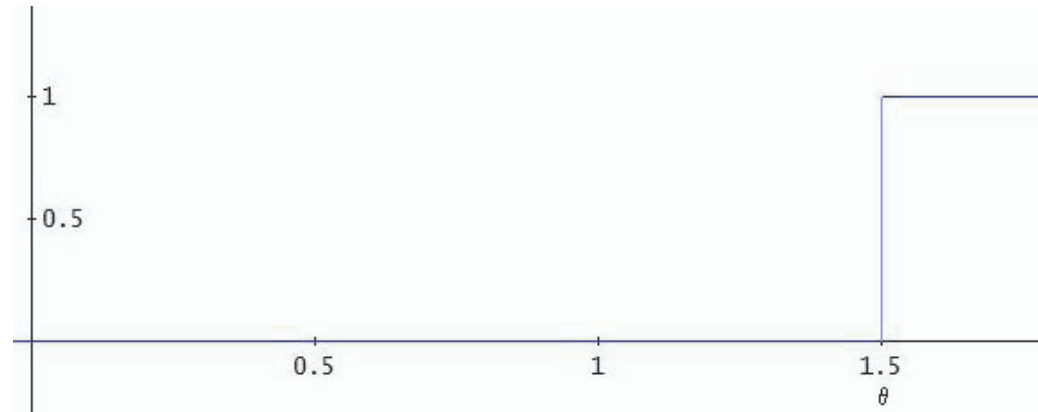
An ANN is a (directed or undirected) graph $\mathcal{A} = (V, E)$ whose vertices $v \in V$ are called units or neurons, and whose edges E are called (synaptic) connections. V has three subsets: $I \subseteq V$, the input units; $O \subseteq V$, the output units; $H \subset V$, the (*hidden*) units, s.t. $H \cap I = H \cap O = \emptyset$.



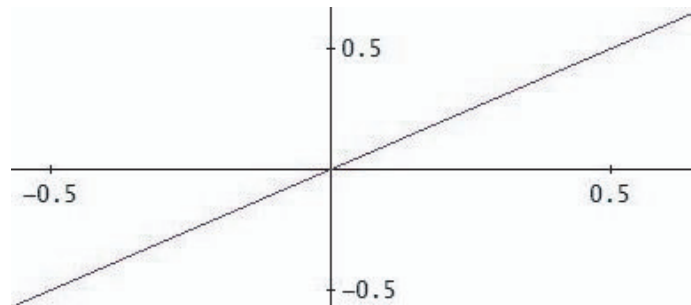
Vertices and edges of the graph are labeled: the edge labels, known as the *weights*, are scalars $w \in \mathbb{R}$; vertex labels are real-valued functions, known as the *activation functions*.

Typical activation functions $f(.) : \mathbb{R} \rightarrow \mathbb{R}$

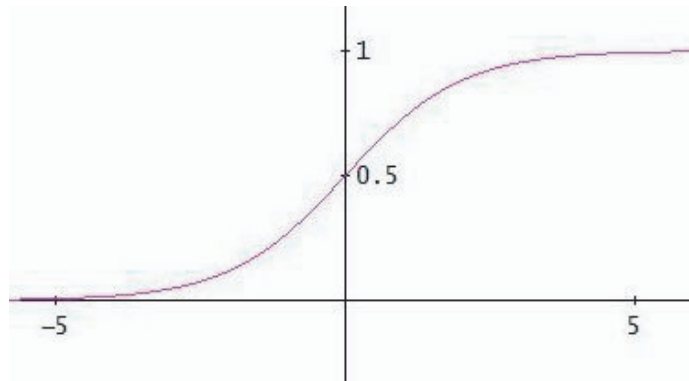
- ▶ Step functions, a.k.a. Threshold Logic Units (TLU):



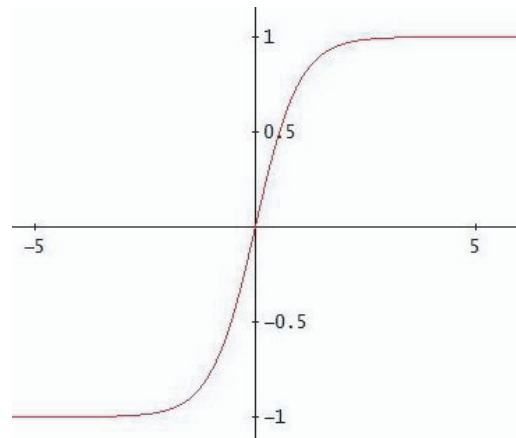
- ▶ Linear functions $f(a) = a$:



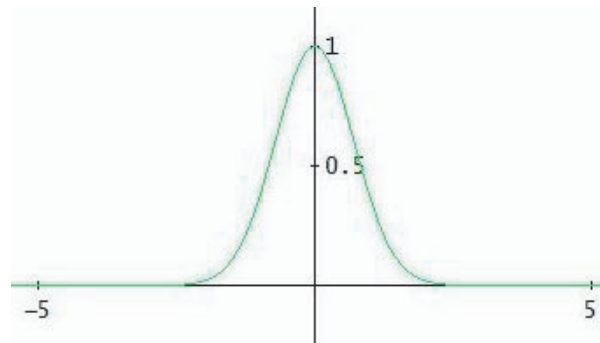
- ▶ (Logistic) sigmoids $f(a) = 1/(1 + e^{-a})$, $0 < f(a) < 1$:



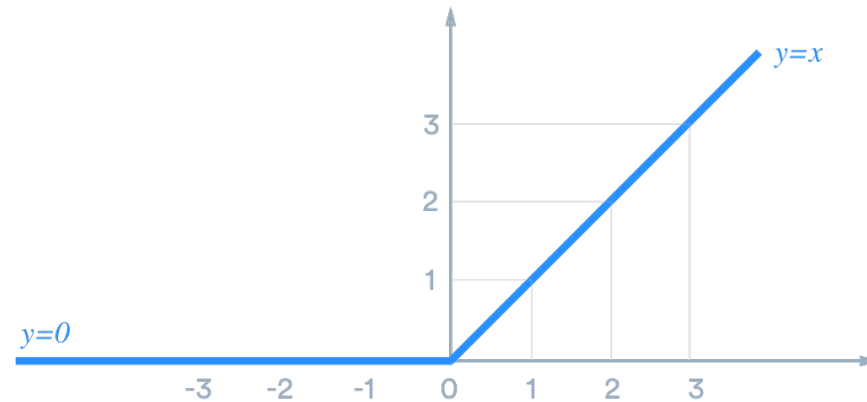
- ▶ Hyperbolic tangent sigmoid $f(a) = \tanh(a)$, $-1 < f(a) < 1$:



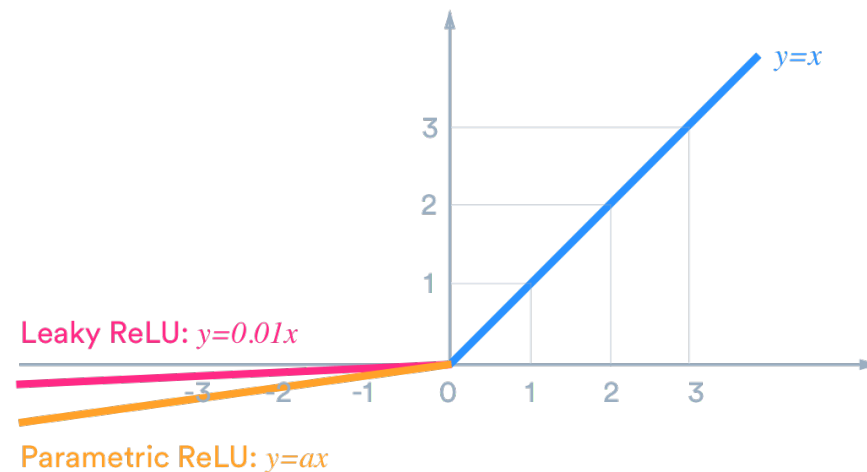
- ▶ Gaussian $f(a) = N(a; 0, 1)$:



- ▶ Rectifier Linear Units (ReLU) $f(a) = \max(0, a)$:

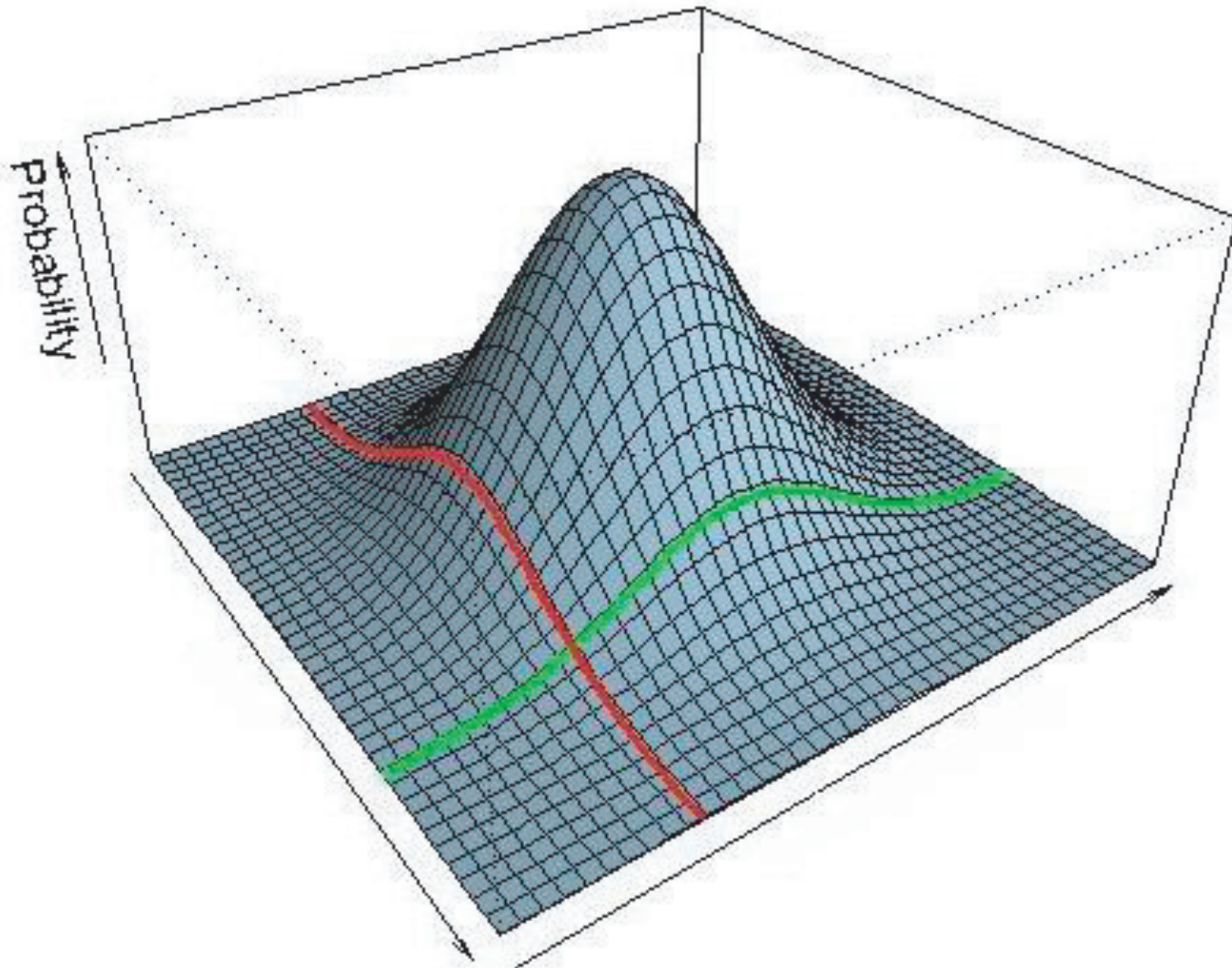


- ▶ Leaky ReLU $f(a) = \max(\lambda a, a)$, where (say) $\lambda \in (-0.1, 0.1)$:

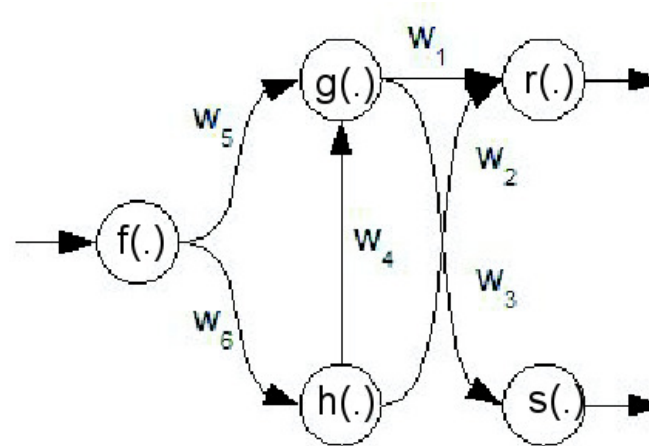


Typical activation functions $f(.) : \mathbb{R}^d \rightarrow \mathbb{R}$

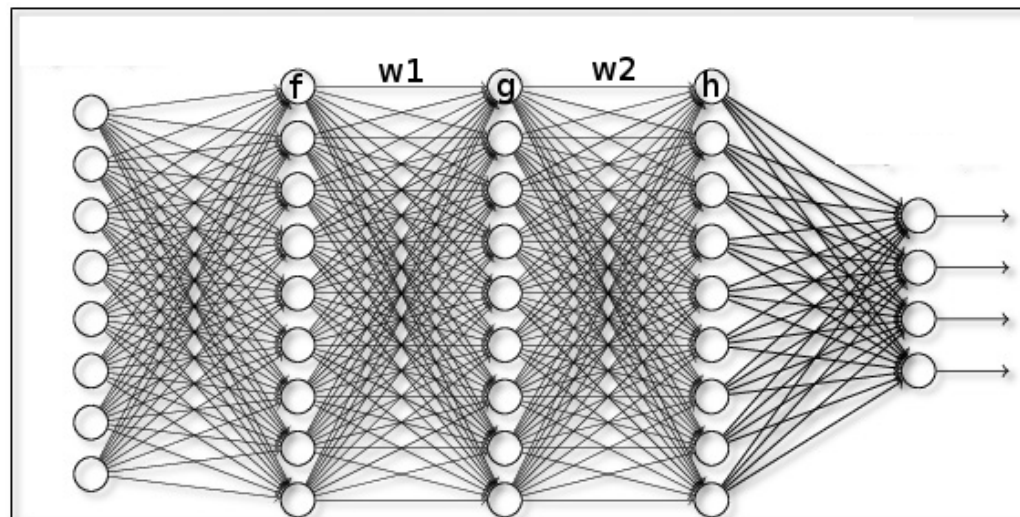
- Scalar fields, such as multivariate Gaussian kernels
 $f(\underline{a}) = N(\underline{a}; \underline{\mu}, \underline{\Sigma})$:



The **architecture**, or topology, of an ANN is thus specified by (V, E, I, O, H) and by the labels $\{w\}$ and $\{f(\cdot)\}$:

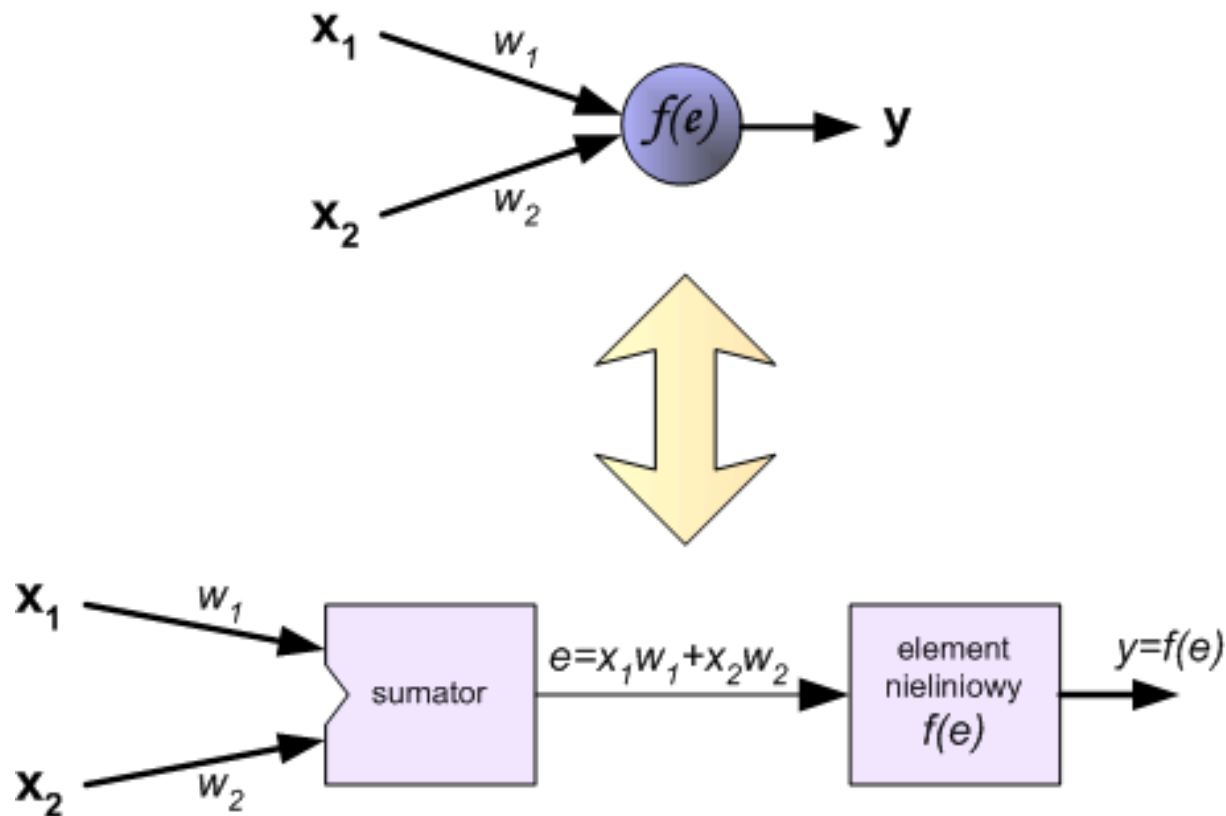


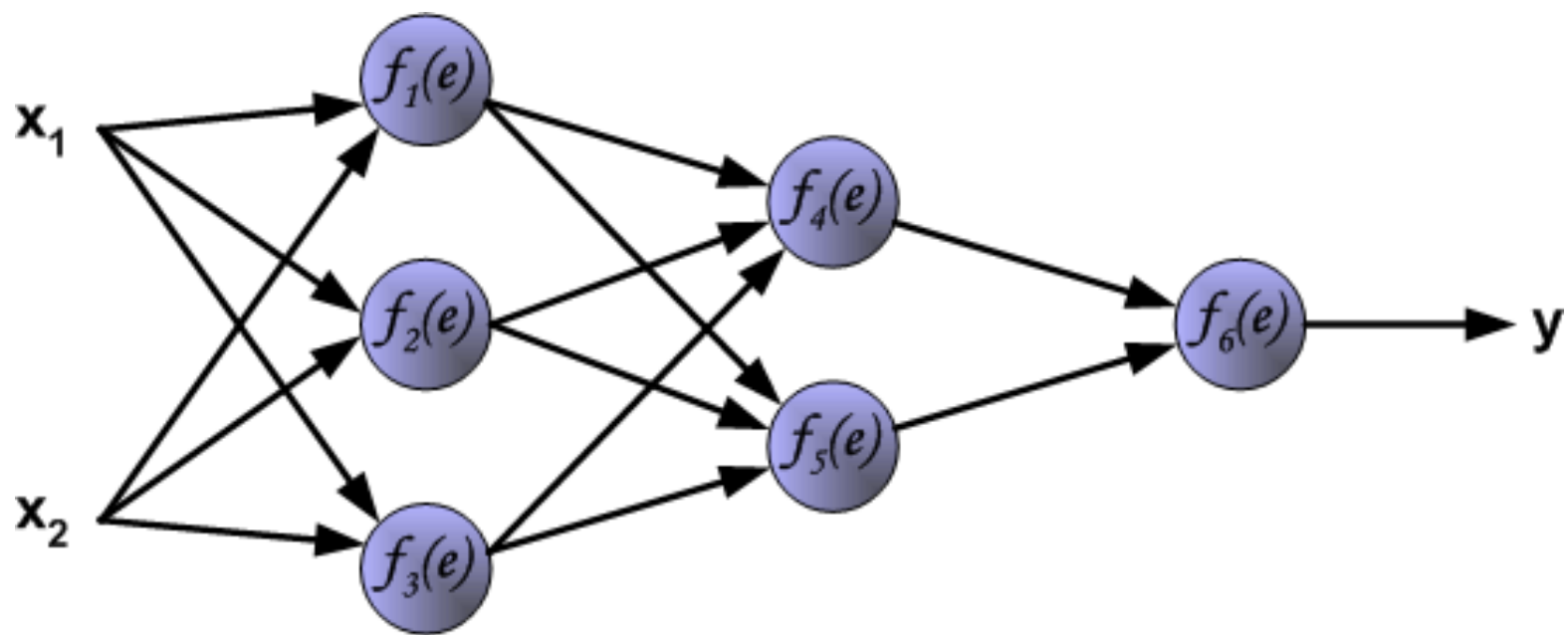
Bigger instance:

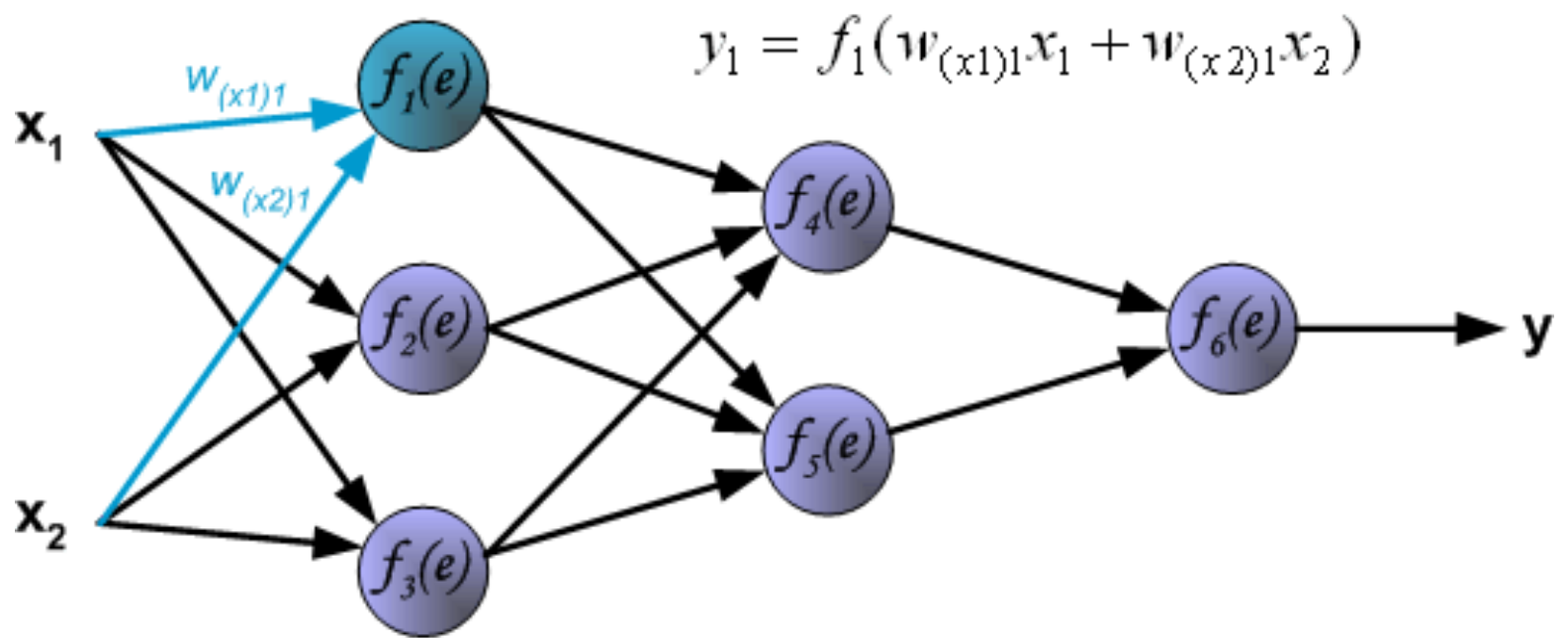


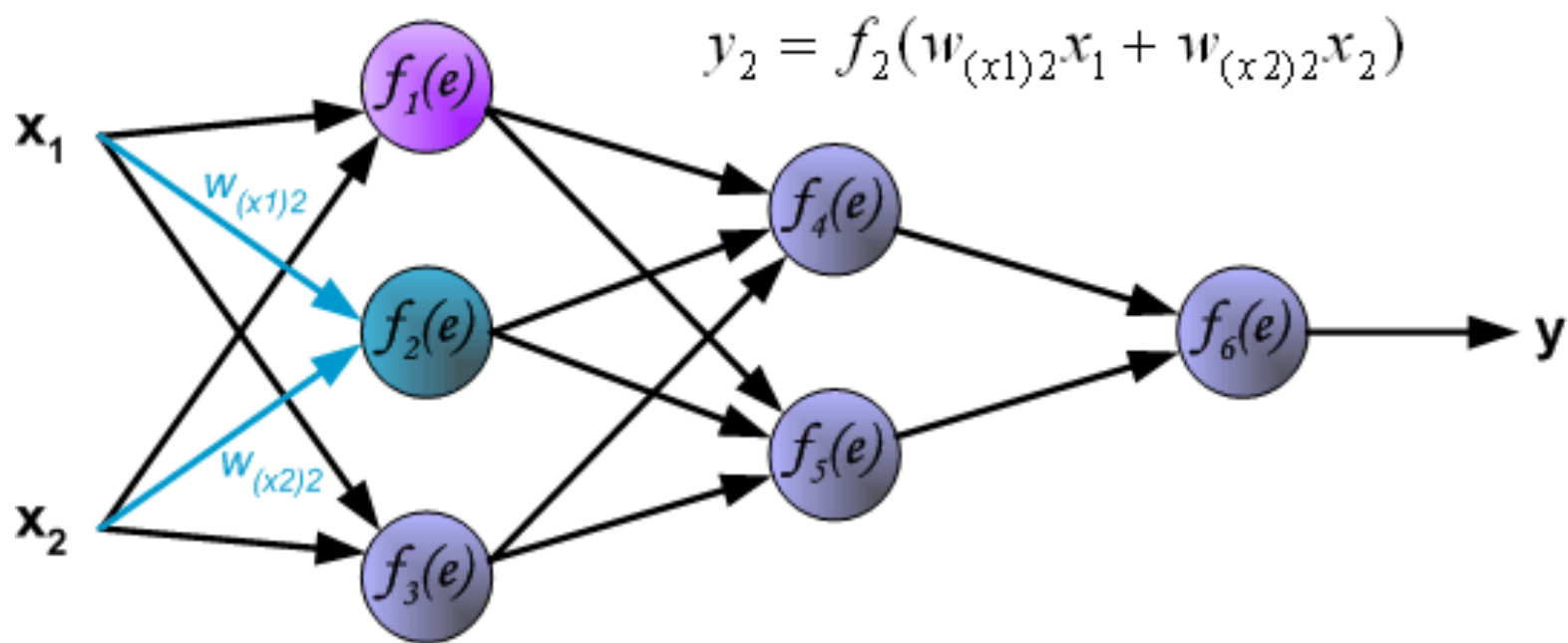
ANN definition #2: dynamics

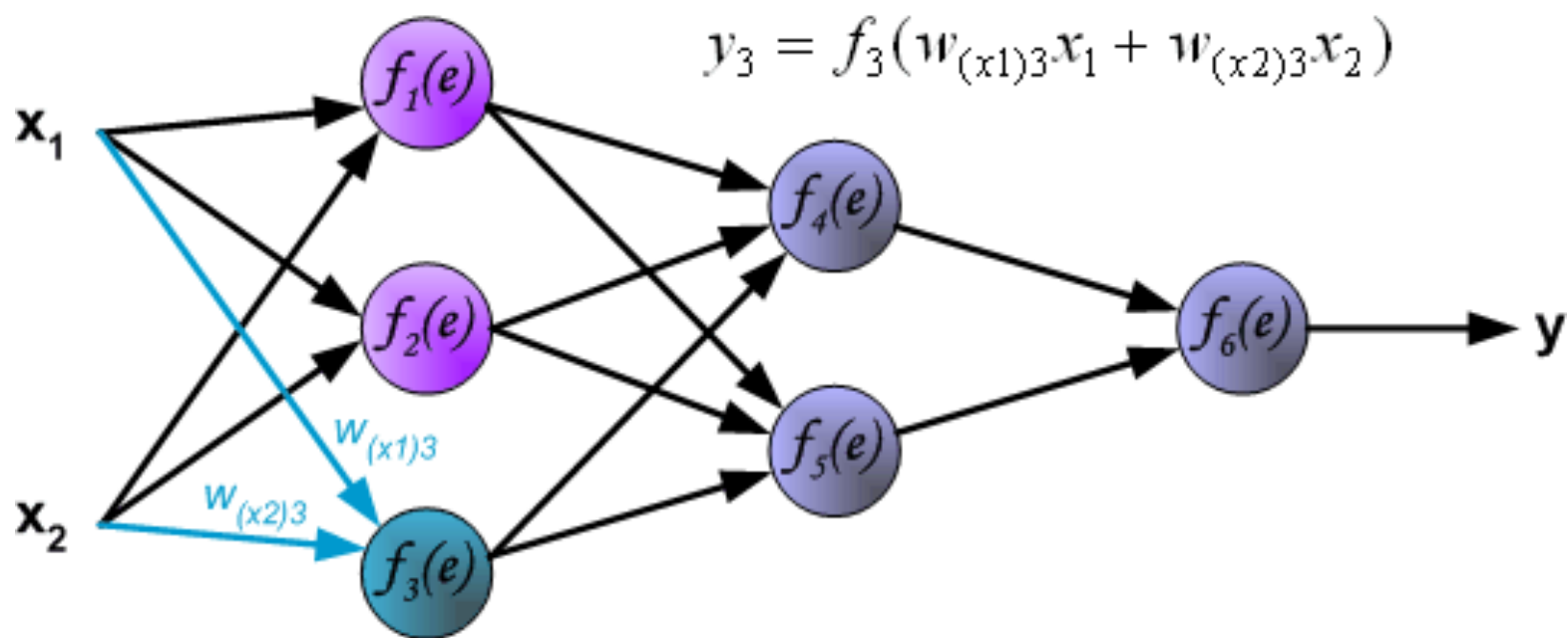
The **dynamics** of an ANN specifies how the signal propagates through the network, that is to say “how the ANN becomes alive”.

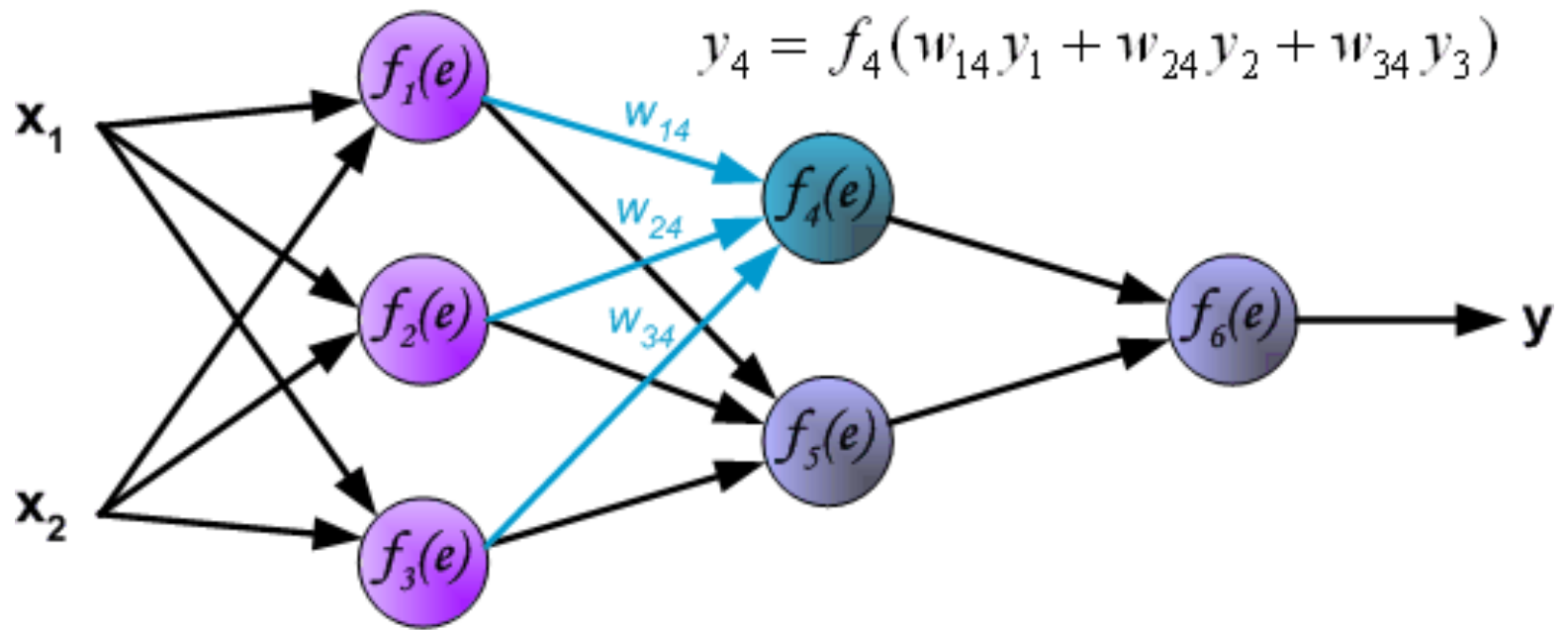


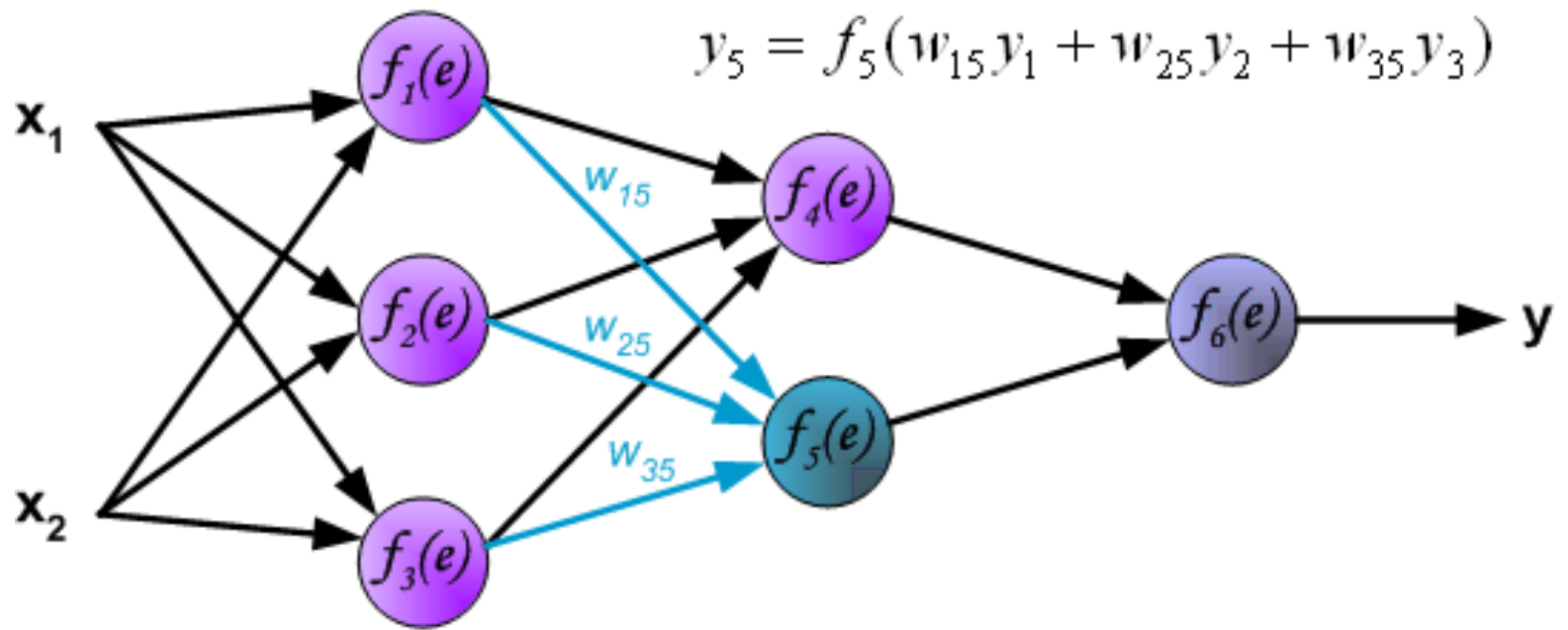


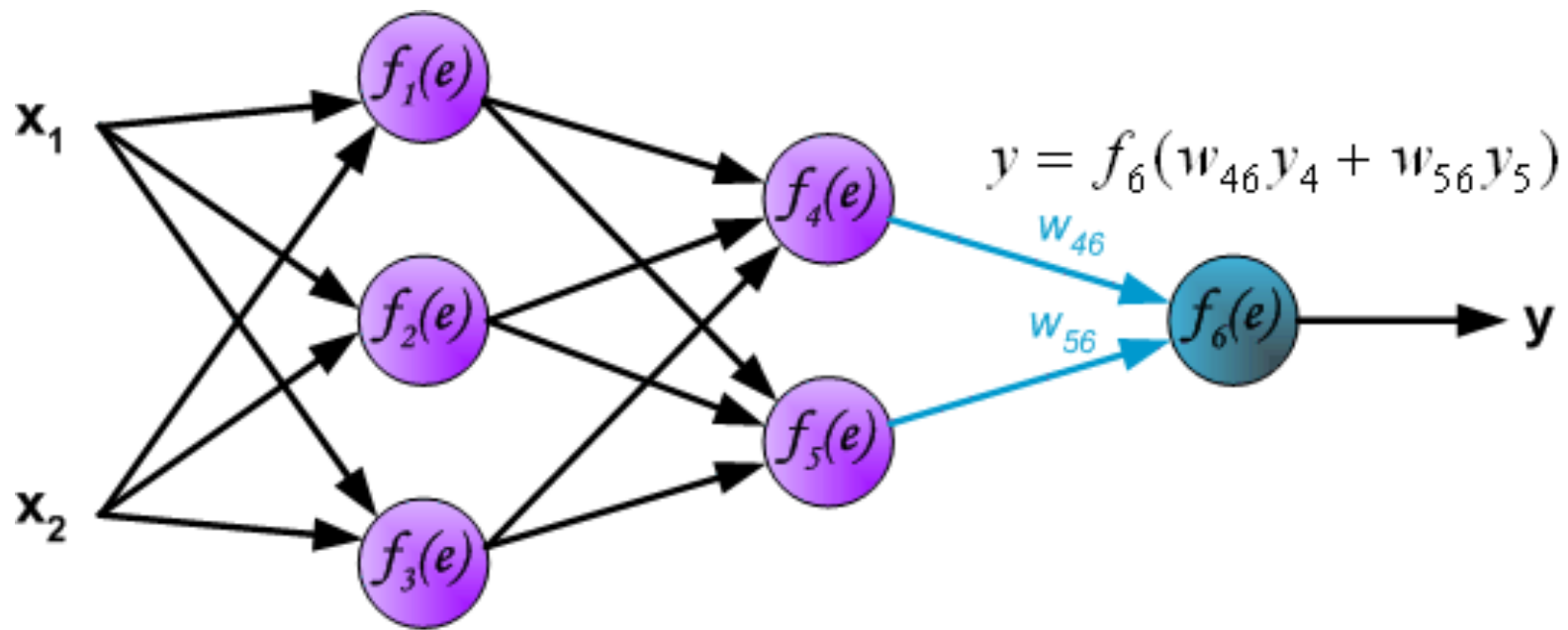












Formally:

1. the ANN is fed with an input $\underline{x}$, $\underline{x} = (x_1, \dots, x_d)$. To this end, the ANN must have d input units, $|I| = d$ where x_i is the input for i -th input unit. The input units do not transform the signal, but they forward it to other neurons along their outgoing connections.
2. in traversing a connection having weight w , the signal o undergoes a transformation, namely it is multiplied by w .

3. let j be a generic unit in $H \cup O$. It receives as many signals $w_1 o_1, \dots, w_n o_n$ as the number of incoming connections. There are two cases:

3.1 the activation function associated with the unit j is scalar, i.e. $f : \mathbb{R} \rightarrow \mathbb{R}$. In this case the input (i.e., activation) a_j to j is the weighted sum of all incoming contributions:

$$a_j = \sum_{i=1}^n w_i o_i \quad (1)$$

and the output o_j from the unit is obtained as $o_j = f(a_j)$ (which, in turn, is propagated forward along the outgoing connections to other neurons).

3.2 the activation function associated with the unit j is $f : \mathbb{R}^n \rightarrow \mathbb{R}$. In this case, $f(\cdot)$ is applied to $\underline{a} = (w_1 o_1, \dots, w_n o_n)$ such that $o_j = f(\underline{a})$, e.g. a Gaussian evaluated on $\underline{a}$.

4. the signal propagates this way through the whole ANN, up to the output units.

Let k be an output unit (i.e., $k \in O$). Then, the value o_k is the k -th output of the ANN. If the latter has m output units, the ANN “output” is the overall, m -dimensional vector $Y = (y_1, \dots, y_m)$ where $y_k = o_k$.

Note: the **time trigger** (clock) of ANNs depends on the specific family of networks at hand.

Remark: the ANN may also be seen as a particular **computation paradigm** (like an analog computer, or Turing machine) whose units are simple analog processors. The ANN topology specifies the hardware architecture, the value of the connection weights w is the “software”, while the ANN dynamics (the living machine) represents the running process.

ANN definition #3: learning (and generalization)

The definition of an ANN is completed by introducing the main feature of ANNs: the **learning** algorithm.

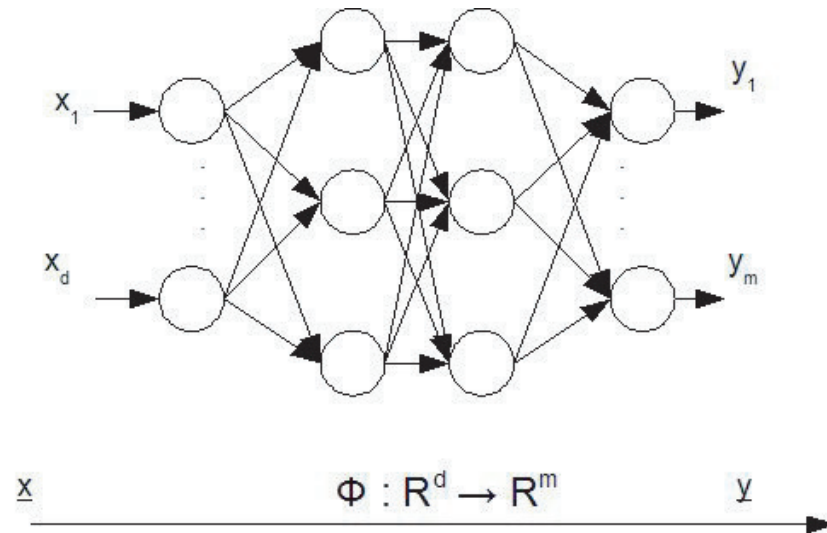
ANNs learn from the examples contained in a **training set**

$\tau = \{\underline{z}_1, \dots, \underline{z}_n\}$, which is a *continuous-valued data sample drawn from an underlying multivariate pdf*. Main learning setups:

- ▶ **supervised** learning: $\tau = \{(\underline{x}_1, \hat{\underline{y}}_1), \dots, (\underline{x}_n, \hat{\underline{y}}_n)\}$
- ▶ **unsupervised** learning: $\tau = \{\underline{x}_1, \dots, \underline{x}_n\}$
- ▶ **semi-supervised** (or, **partially supervised**) learning:
 $\tau = \{(\underline{x}_1, \hat{\underline{y}}_1), \dots, (\underline{x}_n, \hat{\underline{y}}_n), \underline{x}_{n+1}, \dots, \underline{x}_{n+m}\}$
- ▶ **reinforcement learning**: $\tau = \{\underline{x}_1, \underline{x}_2, \dots, (\underline{x}_t, y_t^{(\rho)}), \underline{x}_{t+1}, \dots\}$
where a reinforcement signal $y_t^{(\rho)}$ (either a *penalty* or a *reward*) is given every now and then.

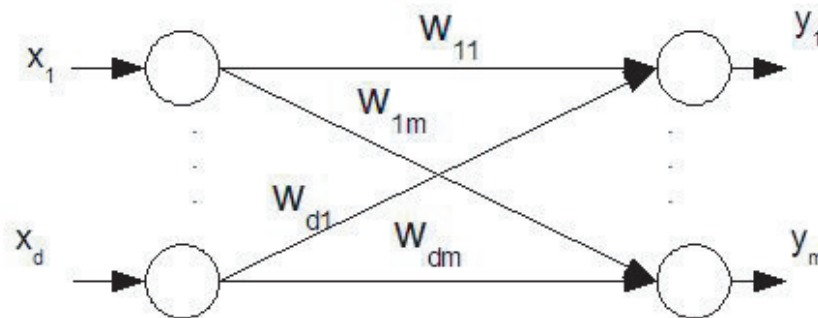
Learning is a process of progressive modification of the connection weights, aimed at inferring the (universal) laws underlying the data. Far from being a bare memorization of the data, the laws learned this way are expected to generalize to new data, previously unseen (**generalization** capability).

Multilayer Perceptron (MLP)



- ▶ *Fully connected* **layered** architecture with **feed-forward** propagation of input signals.
- ▶ Activation functions $f : \mathbb{R} \rightarrow \mathbb{R}$, usually sigmoid and/or linear. All units in a given layer do share the same form of activation function.
- ▶ Dynamics: simultaneous propagation of the signal (with no delays) from the input layer to the output layer, traversing zero, one, or more hidden layers.
- ▶ Learning: supervised, via the gradient method (**backpropagation**).

Special case: Simple Perceptron (SP)



- ▶ Convention: in MLPs, the input layer is not counted (it acts as a *placeholder*); thence, SP is a **1-layer ANN**.
- ▶ Notation: w_{ij} denotes the weight of the connection between j -th input unit and i -th output unit
- ▶ **Computed function**: let $f : \mathbb{R} \rightarrow \mathbb{R}$ be the activation function associated with the output unit(s). Then:

$$y_i = f(a_i) = f\left(\sum_{j=1}^d w_{ij}x_j\right) \quad (2)$$

- ▶ In particular, if f is linear then $y_i = \sum_j w_{ij}x_j = \underline{w}_i^t \underline{x}$ (**Simple Linear Perceptron**)

MLP learning algorithm: Backpropagation

- ▶ Training set: $\tau = \{(\underline{x}_t, \underline{\hat{y}}_t) : t = 1, \dots, n\}$
- ▶ Batch criterion function:
$$C(\tau, w) = C(W) = \frac{1}{2} \sum_{t=1}^n \sum_{i=1}^m (\hat{y}_i - y_i)^2$$
- ▶ **Online criterion function:** $C(W) = \frac{1}{2} \sum_{i=1}^m (\hat{y}_i - y_i)^2$
- ▶ For a generic weight w we apply **gradient descent**:
 $w' = w + \Delta w$ with $\Delta w = -\eta \frac{\partial C}{\partial w}$.
- ▶ Let $f_i(a_i)$ be the activation function associated with the generic i -th unit in the MLP.
- ▶ Assume the MLP has ℓ layers, say L_0 (the **input layer**), $L_1, \dots, L_{\ell-1}$ (the **hidden layers**), and L_ℓ (the **output layer**). We write $k \in L_j$ in order to refer to k -th unit in j -th layer.
- ▶ We aim at calculating $\Delta w = -\eta \frac{\partial C}{\partial w}$ for a generic weight w in the MLP

Case 1: w is in the output layer

Let $w = w_{ij}$, where $j \in L_{\ell-1}$ and $i \in L_{\ell}$. We have:

$$\begin{aligned}\frac{\partial \mathcal{C}}{\partial w_{ij}} &= \frac{\partial}{\partial w_{ij}} \left\{ \frac{1}{2} \sum_{k=1}^m (\hat{y}_k - y_k)^2 \right\} \\ &= \frac{1}{2} \sum_{k=1}^m \frac{\partial}{\partial w_{ij}} (\hat{y}_k - y_k)^2 \\ &= \frac{1}{2} \frac{\partial}{\partial w_{ij}} (\hat{y}_i - y_i)^2 \\ &= -(\hat{y}_i - y_i) \frac{\partial y_i}{\partial w_{ij}}\end{aligned}$$

$$\begin{aligned}\frac{\partial y_i}{\partial w_{ij}} &= \frac{\partial f_i(a_i)}{\partial a_i} \frac{\partial a_i}{\partial w_{ij}} \\ &= f'_i(a_i) \frac{\partial}{\partial w_{ij}} \sum_k w_{ik} o_k \\ &= f'_i(a_i) o_j\end{aligned}$$

In summary, so far we have:

- ▶ $\frac{\partial C}{\partial w_{ij}} = -(\hat{y}_i - y_i) \frac{\partial y_i}{\partial w_{ij}}$
- ▶ $\frac{\partial y_i}{\partial w_{ij}} = f'_i(a_i) o_j$

from which:

$$\Delta w_{ij} = \eta(\hat{y}_i - y_i) f'_i(a_i) o_j$$

Thus, if $i \in L_\ell$ and $j \in L_{\ell-1}$, by defining

$$\delta_i = (\hat{y}_i - y_i) f'_i(a_i)$$

we obtain the following **delta rule**:

$$\Delta w_{ij} = \eta \delta_i o_j \tag{3}$$

where $o_j = f_j(a_j)$.

Remark: this is all we need to train SPs, except for the fact that in SPs we use x_j instead of o_j in the delta rule (plain gradient method).

Case 2: w is in a hidden layer

To fix ideas, let $w = w_{jk}$ where $j \in L_{\ell-1}$ and $k \in L_{\ell-2}$ (still, the results we are presenting hold true in all the hidden layers $L_{\ell-3}, \dots, L_0$). Again, we let $\Delta w_{jk} = -\eta \frac{\partial \mathcal{C}}{\partial w_{jk}}$. We can write:

$$\begin{aligned} \frac{\partial \mathcal{C}}{\partial w_{jk}} &= \frac{\partial}{\partial w_{jk}} \left\{ \frac{1}{2} \sum_{i=1}^m (\hat{y}_i - y_i)^2 \right\} \\ &= \frac{\partial}{\partial o_j} \left\{ \frac{1}{2} \sum_{i=1}^m (\hat{y}_i - y_i)^2 \right\} \frac{\partial o_j}{\partial w_{jk}} \\ &= \left\{ \frac{1}{2} \sum_{i=1}^m \frac{\partial}{\partial o_j} (\hat{y}_i - y_i)^2 \right\} \frac{\partial o_j}{\partial w_{jk}} \\ &= \left\{ - \sum_{i=1}^m (\hat{y}_i - y_i) \frac{\partial y_i}{\partial o_j} \right\} \frac{\partial o_j}{\partial w_{jk}} \end{aligned}$$

Let's calculate $\frac{\partial y_i}{\partial o_j}$.

$$\begin{aligned}
\frac{\partial y_i}{\partial o_j} &= \frac{\partial f_i(a_i)}{\partial a_i} \frac{\partial a_i}{\partial o_j} \\
&= f'_i(a_i) \frac{\partial}{\partial o_j} \sum_j w_{ij} o_j \\
&= f'_i(a_i) w_{ij}
\end{aligned}$$

from which, since $\frac{\partial C}{\partial w_{jk}} = \left\{ -\sum_{i=1}^m (\hat{y}_i - y_i) \frac{\partial y_i}{\partial o_j} \right\} \frac{\partial o_j}{\partial w_{jk}}$, we obtain:

$$\begin{aligned}
\frac{\partial C}{\partial w_{jk}} &= \left\{ -\sum_{i=1}^m (\hat{y}_i - y_i) f'_i(a_i) w_{ij} \right\} \frac{\partial o_j}{\partial w_{jk}} \\
&= - \left(\sum_{i=1}^m w_{ij} \delta_i \right) \frac{\partial f_j(a_j)}{\partial a_j} \frac{\partial a_j}{\partial w_{jk}} \\
&= - \left(\sum_{i=1}^m w_{ij} \delta_i \right) f'_j(a_j) o_k
\end{aligned}$$

Thus far, $\Delta w_{jk} = -\eta \frac{\partial C}{\partial w_{jk}}$ and $\frac{\partial C}{\partial w_{jk}} = -(\sum_{i=1}^m w_{ij} \delta_i) f'_j(a_j) o_k$.
 Thence, we can write:

$$\Delta w_{jk} = \eta \left(\sum_{i=1}^m w_{ij} \delta_i \right) f'_j(a_j) o_k \quad (4)$$

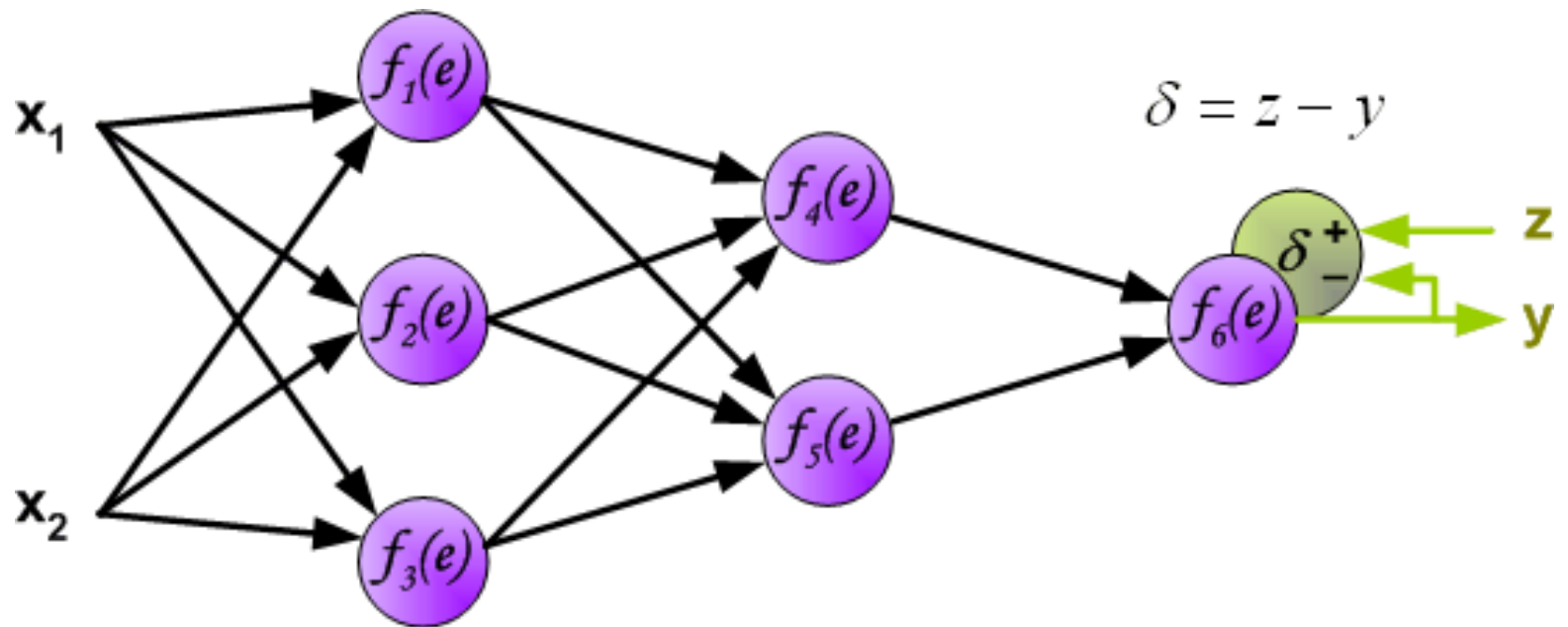
By defining $\delta_j = (\sum_{i=1}^m w_{ij} \delta_i) f'_j(a_j)$, the following *delta rule* is obtained:

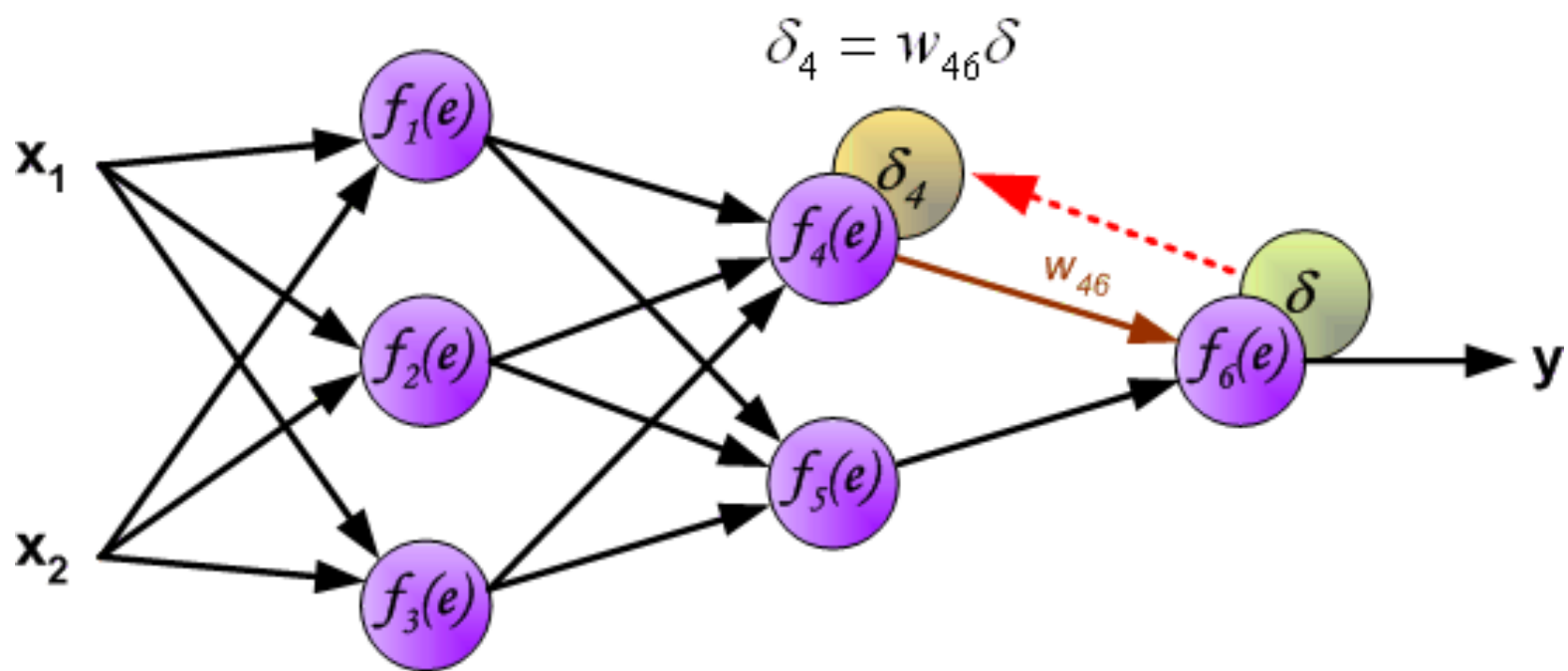
$$\Delta w_{jk} = \eta \delta_j o_k \quad (5)$$

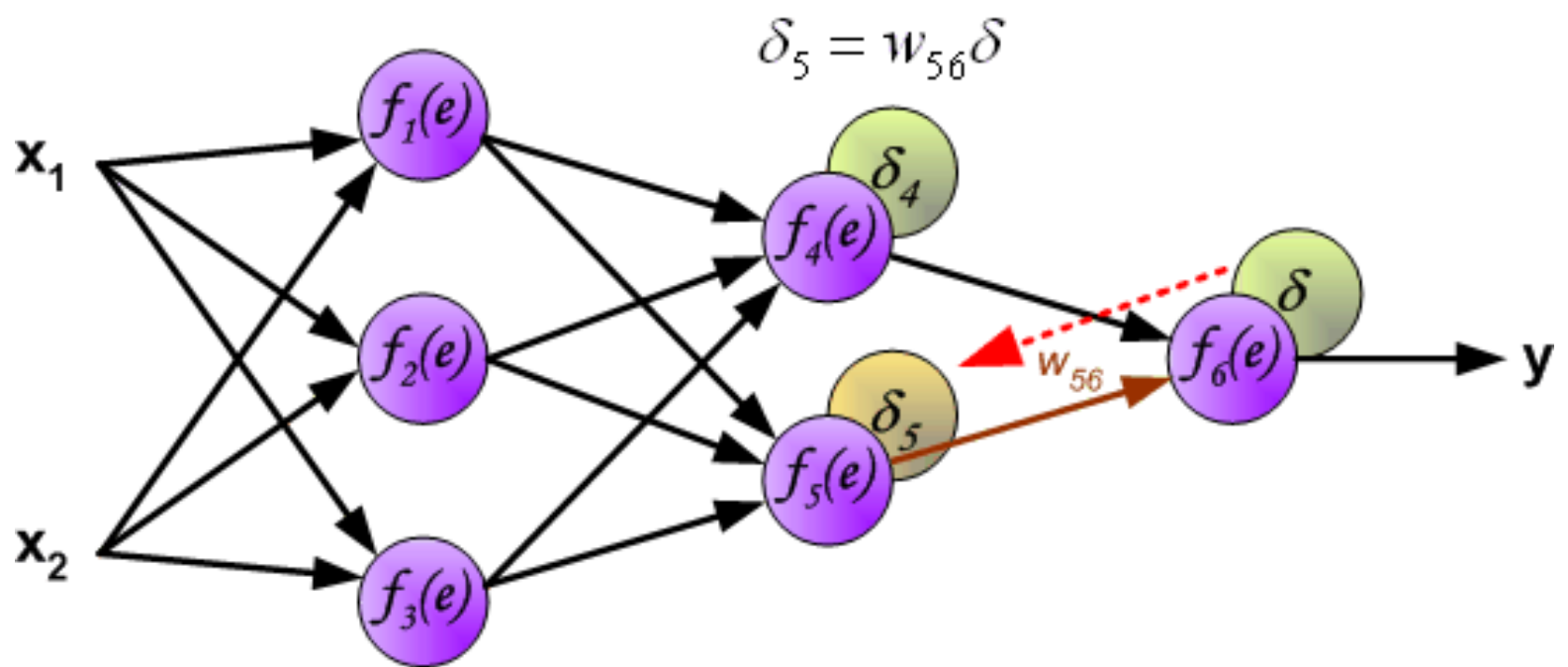
which is in the same form as in case 1. More generally, given any weight w_{jk} in any layer, the **generalized delta-rule** $\Delta w_{jk} = \eta \delta_j o_k$ can be applied, provided that we define:

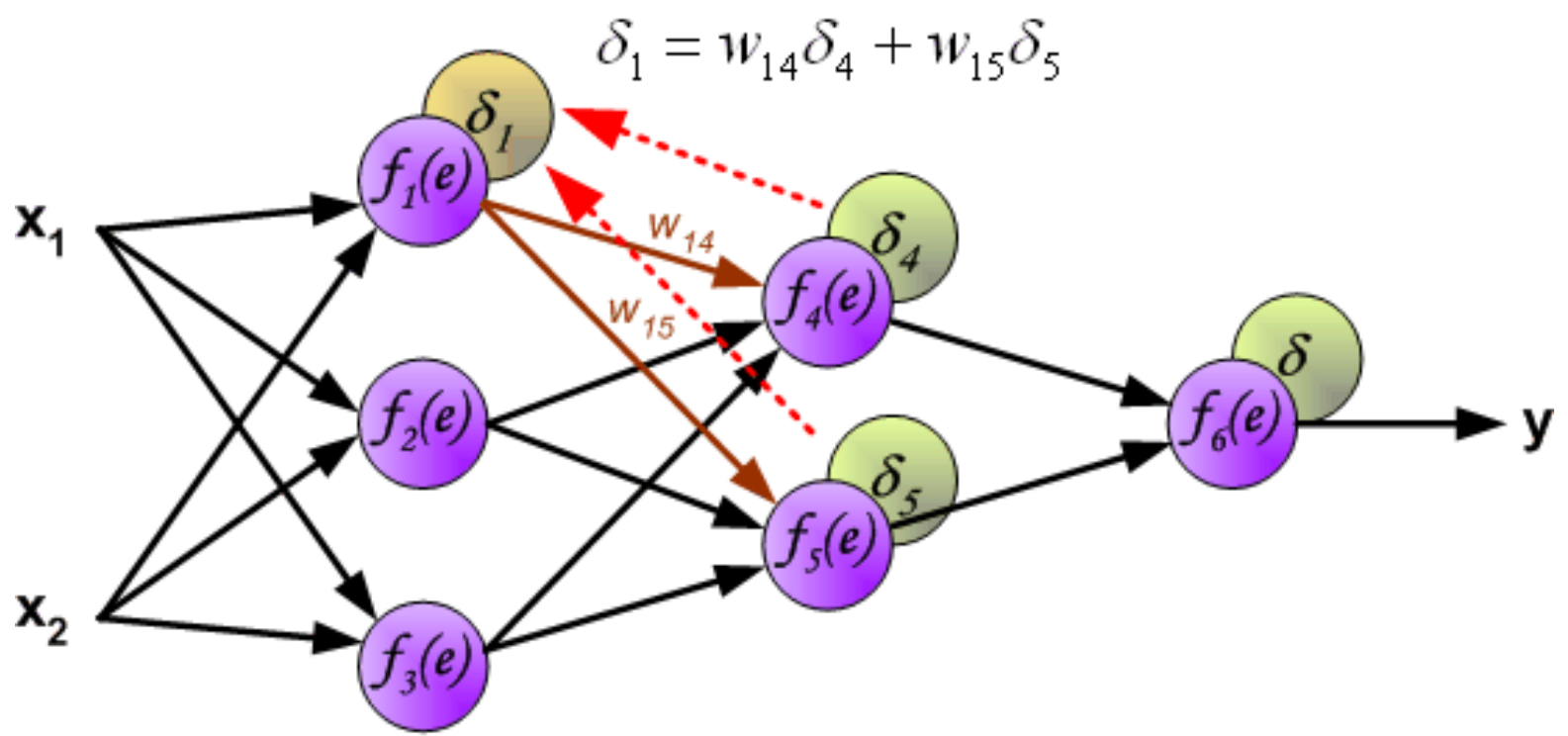
$$\delta_j = \begin{cases} (\hat{y}_j - y_j) f'_j(a_j) & \text{if } j \in L_\ell \\ (\sum_{i \in L_{k+1}} w_{ij} \delta_i) f'_j(a_j) & \text{if } j \in L_k \text{ where } k = \ell - 1, \dots, 0 \end{cases}$$

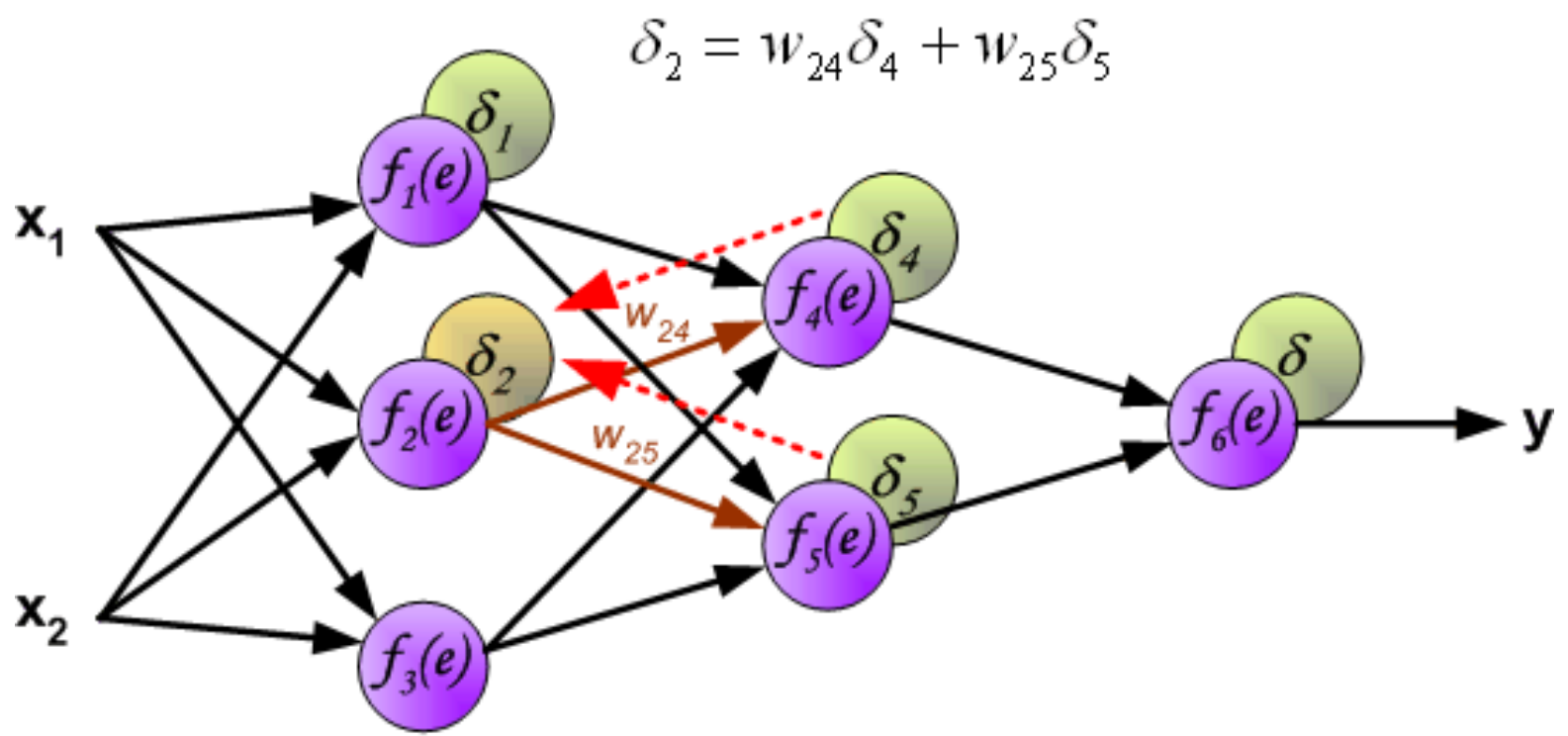
Learning is iterated for a number of consecutive **epochs** on the training set (an epoch is a cycle of application of the delta-rule over all the data in τ)

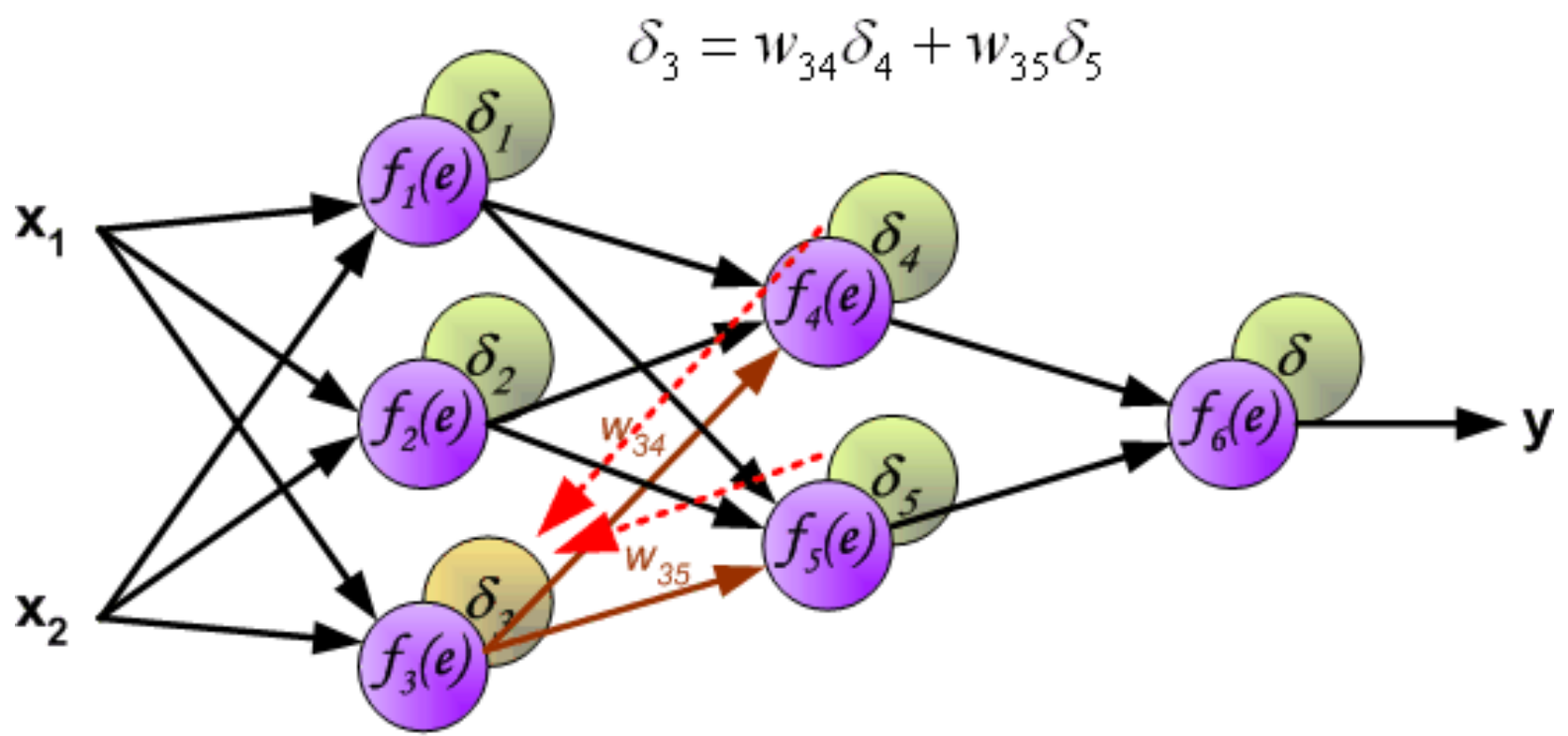






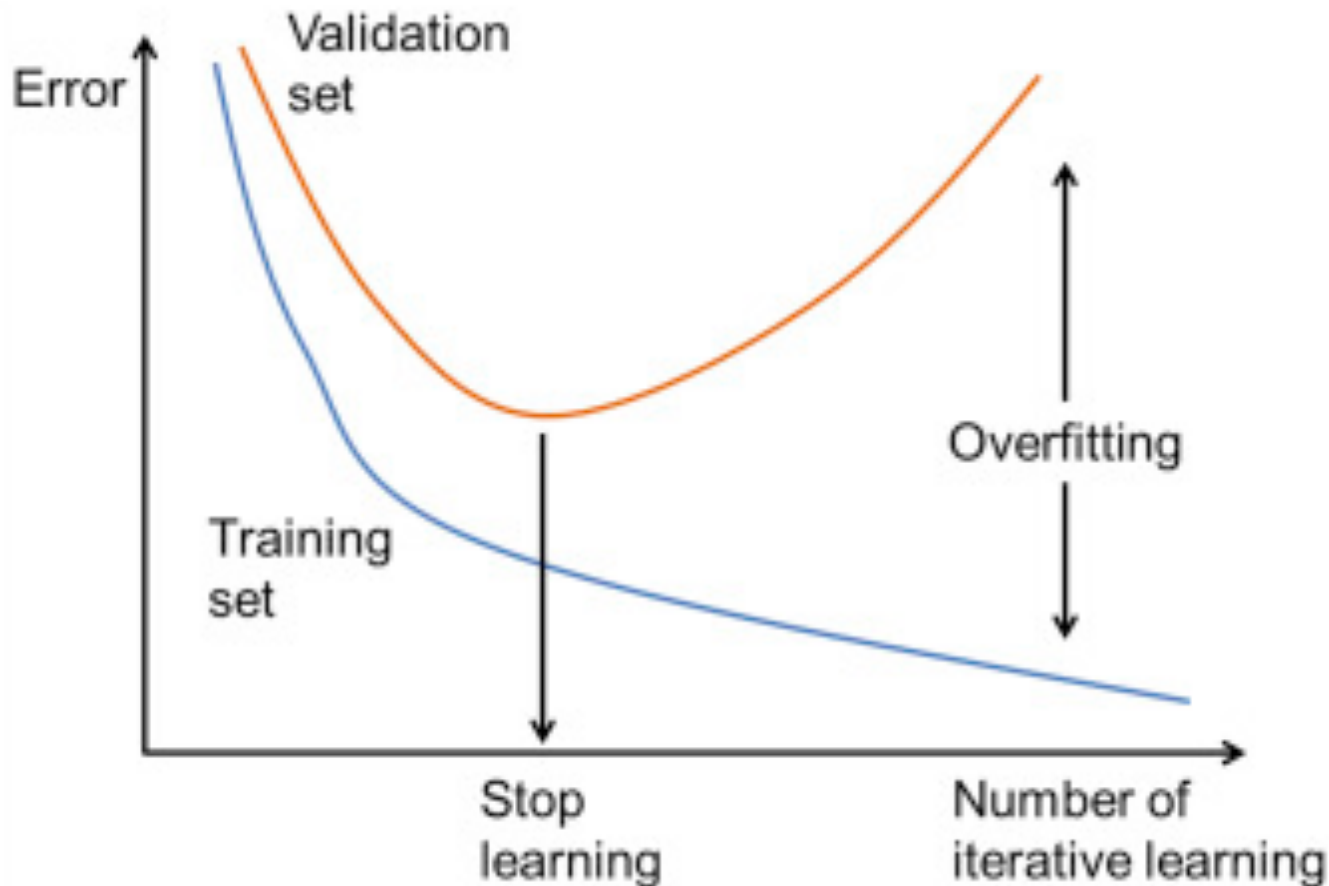






Stopping Criteria

1. upon a fixed number of epochs
2. once the error C falls below a given threshold: $C < \epsilon$
3. once the gain ΔC falls below a given threshold: $\Delta C < \epsilon$
4. evaluating (via cross-validation) the generalization capability:

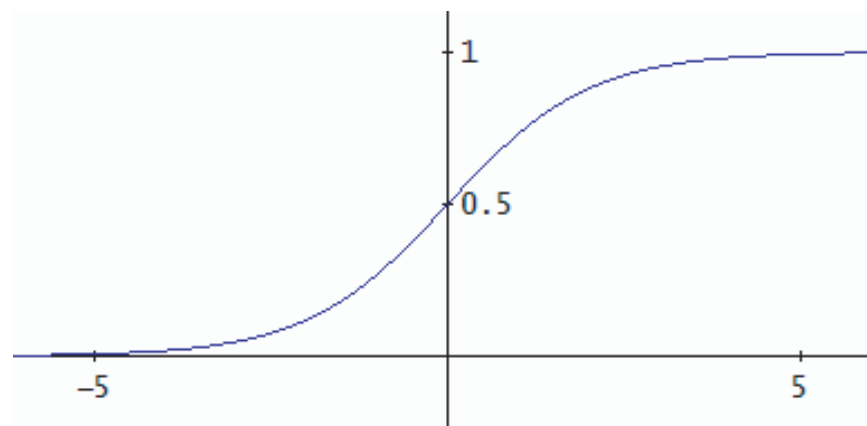


Learning curve: error measured on the training set.

Generalization curve: error measured on the validation set.

Practical Issues and Some Insights

1. The **weights need random initialization**, uniformly over a small zero-centered range ($w_{ij} \in [-\varepsilon, \varepsilon]$) in order to avoid the **saturation** of sigmoids:



2. For the same reason, as well as for numerical stability issues, input values ($x_1, x_2, \dots, x_d$) must be **normalized**. For instance: $x_i \in [0, 1]$, or $x_i \in [-1, 1]$. A typical choice that guarantees values in the $[-1, 1]$ range with null mean is: for each component $i = 1, \dots, d$, subtract its average value and divide by the maximum absolute value of i -th component.
3. If sigmoids are used in the output layer, then also the target outputs shall be normalized to $[0, 1]$.

4. **Regularization:** learning is modified in order to improve the generalization capabilities of the MLP. An idea is to force the complexity of the machine to be as small as possible (i.e., to limit the VC-dimension):

4.1 **weight-sharing:** some connections are forced to share the same weight value w , and Δw is computed by averaging over the different values yielded by the delta-rule over the corresponding, individual instances of w

4.2 **weight-decay:** numerically smaller weights entail simpler solutions:

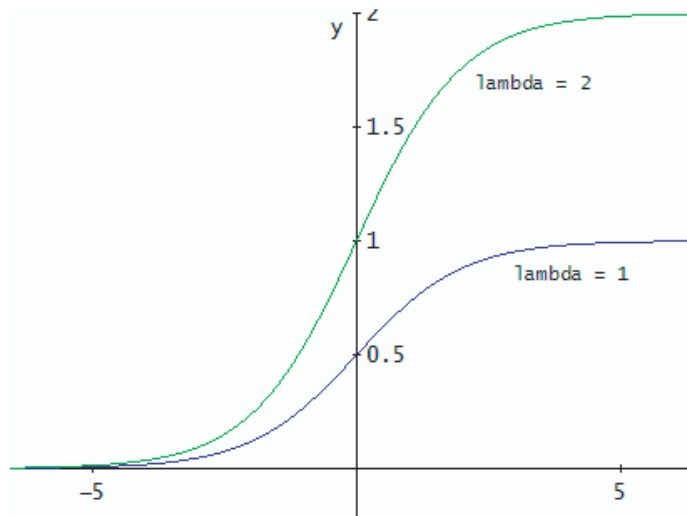
$$C = \frac{1}{2} \sum_i (\hat{y}_i - y_i)^2 + \frac{\alpha}{2} \sum_{i,j} (w_{ij})^2 \quad (6)$$

where $\frac{\alpha}{2} \sum_{i,j} (w_{ij})^2$ is the **regularization term**. For a generic w we have $\Delta w = -\eta \frac{\partial C}{\partial w} = -\eta \frac{\partial}{\partial w} \left\{ \frac{1}{2} \sum_i (\hat{y}_i - y_i)^2 \right\} - \eta \alpha w$ i.e. in addition to the usual Δw due to the delta rule, learning w requires decreasing it by a fraction $\eta \alpha$ of its own.

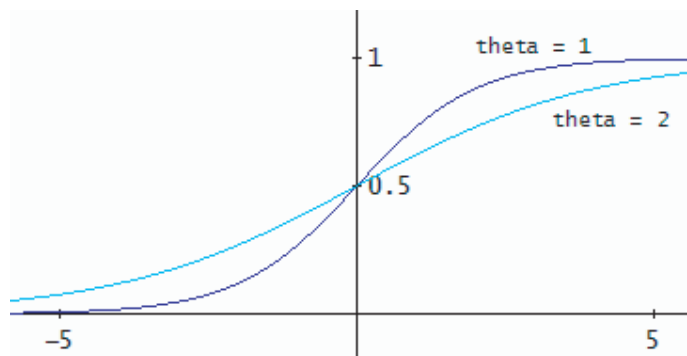
5. We can use more flexible activation functions:

$$f(a) = \frac{\lambda}{1 + e^{-(a-b)/\theta}}$$

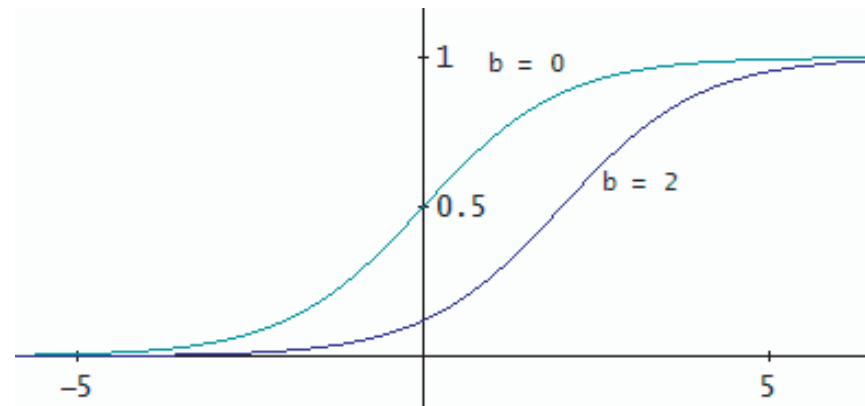
► λ is the **amplitude**:



► θ is the **smoothness** (high impact on the derivative):



- b is the **bias** (or, offset):



There are (gradient-descent) algorithms for learning λ, θ, b . In particular, the bias may be learned just as a regular weight by means of a **diagonalization unit** having constant output set to 1.

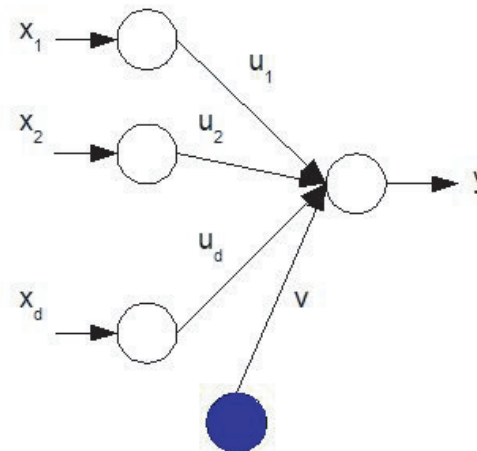


Figure: Bias

6. Learning may be made more stable by including a **momentum term** (or, *inertia*) in the delta rule:

$$\Delta w(t+1) = -\eta \frac{\partial C}{\partial w(t)} + \rho \Delta w(t) \quad (7)$$

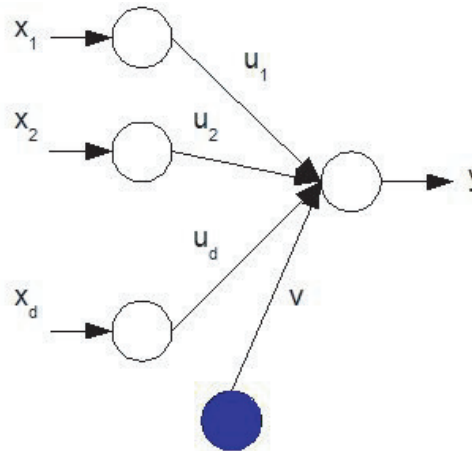
where $\rho \in (0, 1)$ is the *momentum rate*

Main supervised learning tasks

A supervised MLP learns a transformation $\phi : \mathcal{R}^d \rightarrow \mathcal{R}^m$ which, at least in principle, may be applied to:

- ▶ **function approximation:** $\underline{y} = \phi(\underline{x})$
- ▶ **regression** (linear or non-linear): $\underline{y} = \phi(\underline{x}) + \underline{\epsilon}$, where $\underline{\epsilon}$ is the multivariate (Gaussian) noise
- ▶ **pattern classification:** $\underline{y} = (g_1(\underline{x}), \dots, g_c(\underline{x}))$ (i.e., the vector of discriminant functions for classes $\omega_1, \dots, \omega_c$)

In 2-class classification, in particular:



where $y = g(\underline{x}) = \underline{u}^t \underline{x} + v$. In practice, the Simple Linear Perceptron is a **linear discriminant (Widrow-Hoff algorithm:** $\hat{y} = 1$ if the pattern belongs to the class at hand, 0 otherwise). It is seen that if the net has c outputs (for the c classes), the bias values will be different for each output, such that $g_i(\underline{x}) = \underline{u}_i^t \underline{x} + v_i$, for each $i = 1, \dots, c$.

If we train a MLP on the same training set (e.g., labeled with $\hat{y}_i = 0/1$, or $\hat{y}_i = -1/1$) we obtain **non-linear** discriminant functions (if the output layer is linear, then we have a **Generalized Linear Discriminant**), having flexible (not necessarily **convex** and **connected**) decision regions.

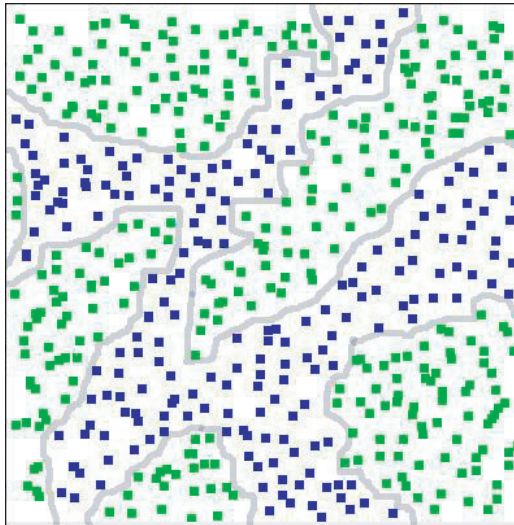


Figure: Nonlinear separation surfaces

Q: how flexible?

A: very! In fact the following theorem holds:

Universality of MLPs (Cybenko, Lippmann, 1989): a MLP with an hidden layer of sigmoid functions and a linear output layer is a universal machine. Formally: let $\epsilon \in \mathbb{R}^+$ be any arbitrarily small positive constant. $\forall \varphi : \mathbb{R}^d \rightarrow \mathbb{R}^m$ s.t. φ is continuous and limited over a closed support $X \subset \mathbb{R}^d$, then a two (hidden-sigmoid, linear-output) layers MLP $\phi(\cdot)$ exists such that $\|\phi(\cdot) - \varphi(\cdot)\|_{X,\infty} < \epsilon$ (where $\|\cdot\|_{X,\infty}$ is the Chebyshev norm over X).

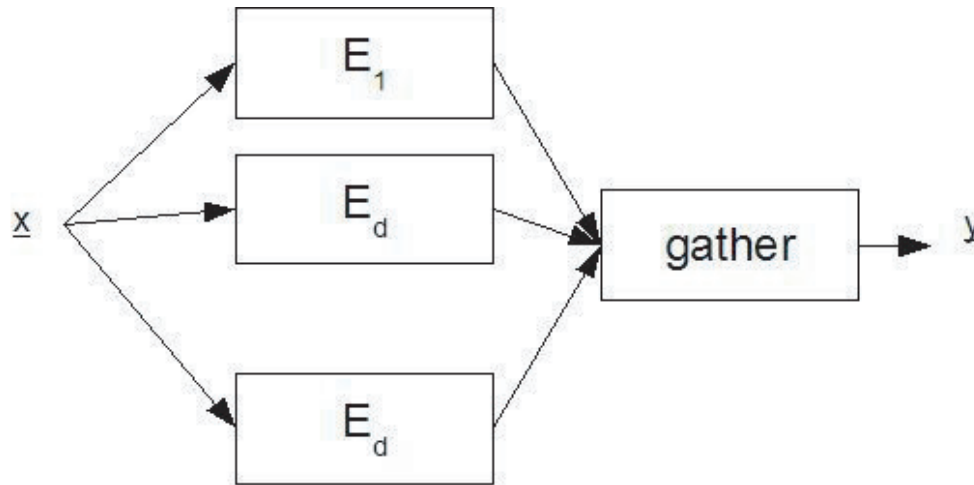
Remarks: universality says that "exists a MLP...", but it does not say:

- ▶ how many neurons such a MLP has;
- ▶ what is the correct value of its weights ;
- ▶ if and how it is possible to *learn* the correct weights.

Nonetheless, universality proofs the flexibility and the modeling capabilities of MLPs, e.g. **when it comes to heavily nonlinear discriminant functions**

Mixtures of Experts

We can combine k neural modules, or *experts*, $E_1, E_2, \dots, E_k$:



Each E_i is a MLP computing the function $y_i = \varphi_i(\underline{x})$. The overall machine thus realizes:

$$\underline{y} = \sum_{i=1}^k \alpha_i \varphi_i(\underline{x}) \quad (8)$$

where $\alpha_i \in [0, 1]$. α_i is the *credit* assigned from the “gatherer” (or, “gating module”) to E_i over input $\underline{x}$.

(i) The feature space may be partitioned (divide and conquer) into k **disjoint regions** $A_1, A_2, \dots, A_k$ and E_i is specialized (i.e., trained and used) on A_i .

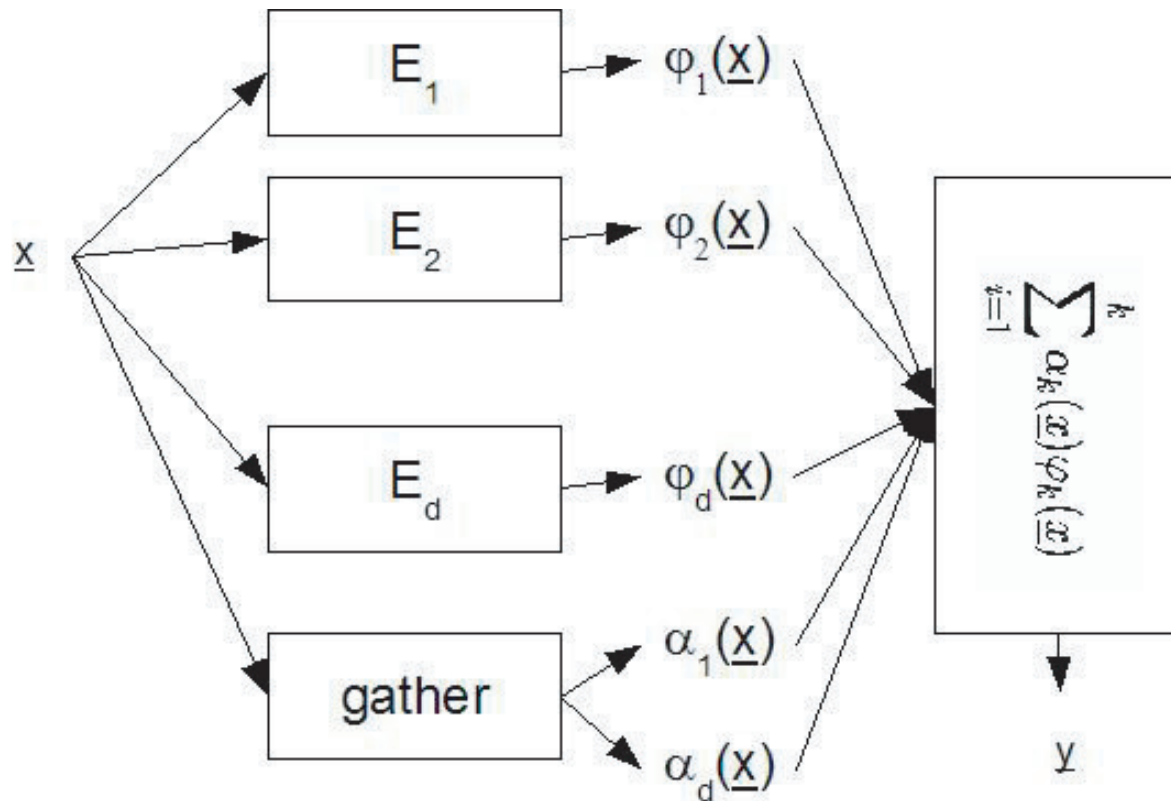
The gatherer assigns credits this way: if $\underline{x} \in A_i$, then $\alpha_i = 1$ and $\alpha_j = 0$ for all $j \neq i$ (during both training and test).

(ii) Instead of fixed regions A_i , we may assume “classes” of credit $\omega_1, \dots, \omega_k$ having pdfs $p_i(\underline{x}|\omega_i)$ (**overlapping regions**) which express the “likelihood” of E_i being competent over any input pattern, e.g. $p(\underline{x}|\omega_i) = N(\underline{x}; \mu, \Sigma)$.

The gatherer assigns credits $\alpha_i = P(\omega_i|\underline{x})$ under the condition $\sum_{j=1}^k \alpha_j = 1$, and $\underline{y} = \sum_{i=1}^k P(\omega_i|\underline{x})\varphi_i(\underline{x})$ (in both training and test).

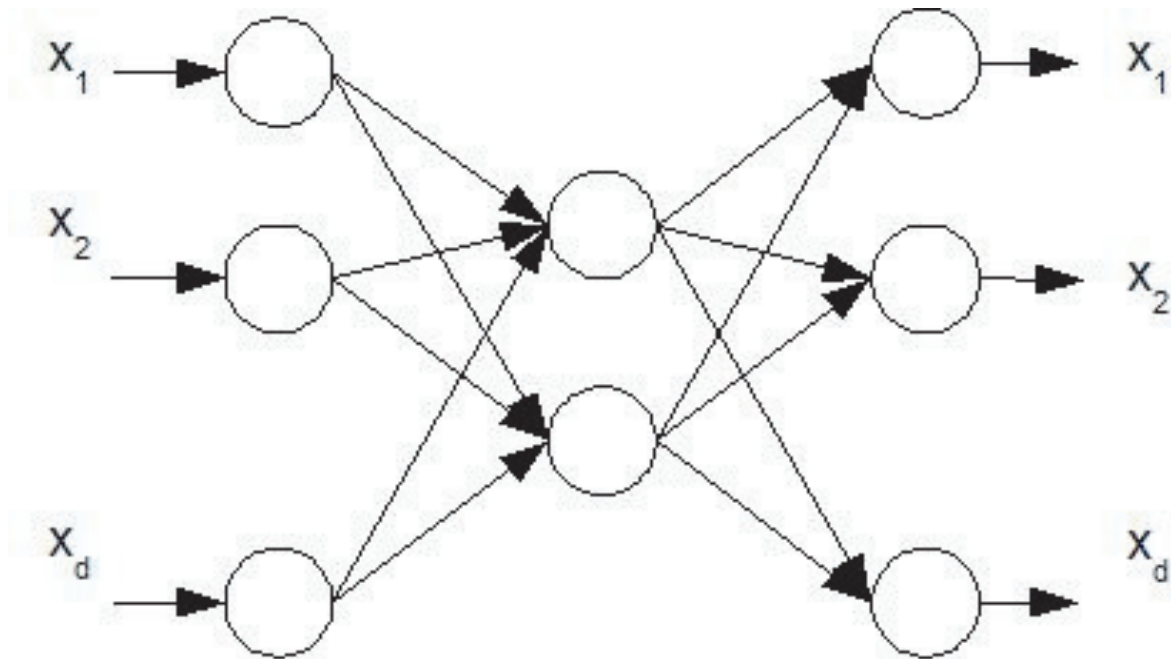
To do that, a model (estimator) of $P(\omega_i|\underline{x})$ is needed. *To this end, Bayesian/statistical techniques are applied:* in so doing, a relevant instance of a neural/statistical *hybrid* emerges.

(iii) Instead of fixing regions or “classes” *a priori*, the machine can **learn the gating function** during training, via a “gating network” (i.e., a referee) whose i -th output is the credit $\alpha_i(\underline{x})$:



The gating network is trained in parallel with the experts, s.t. its output (credit) α_j “weights” the contribution from j -th expert E_j to the overall output, thence α_j affects the backpropagation of derivatives of the criterion function through E_j .

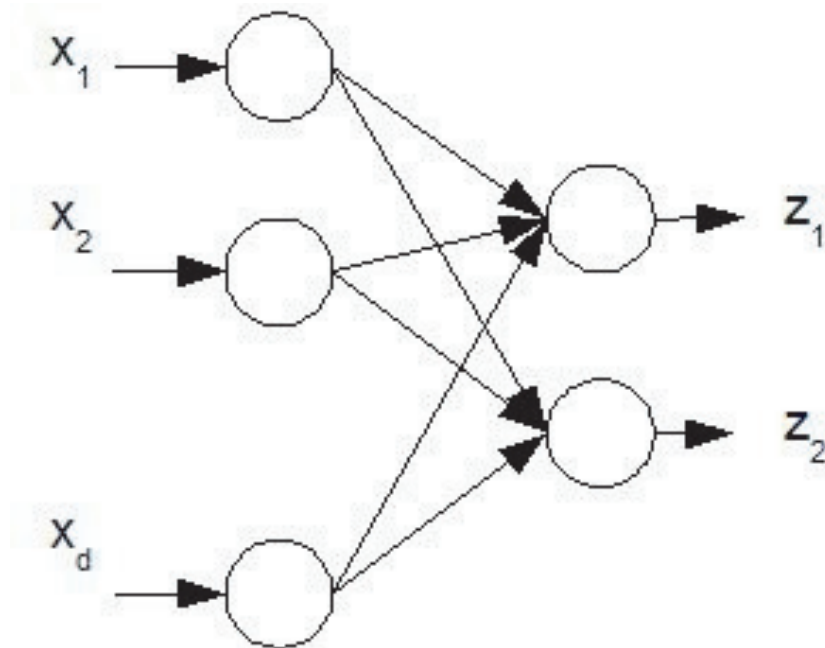
Autoencoder (a.k.a. autoassociative neural net)



It is a MLP which learns to map its inputs onto themselves.
Training set (for BP): $\tau = \{(\underline{x}, \underline{x})\}$. Thus, it may be seen as an example of *unsupervised learning*.

Training

1. The ANN is trained to map $\underline{x}$ onto $\underline{z}$ via BP
2. The hidden layer(s) turn(s) out to be a lower-dimensionality representation of the feature space.
3. Let us remove the upper layer(s):



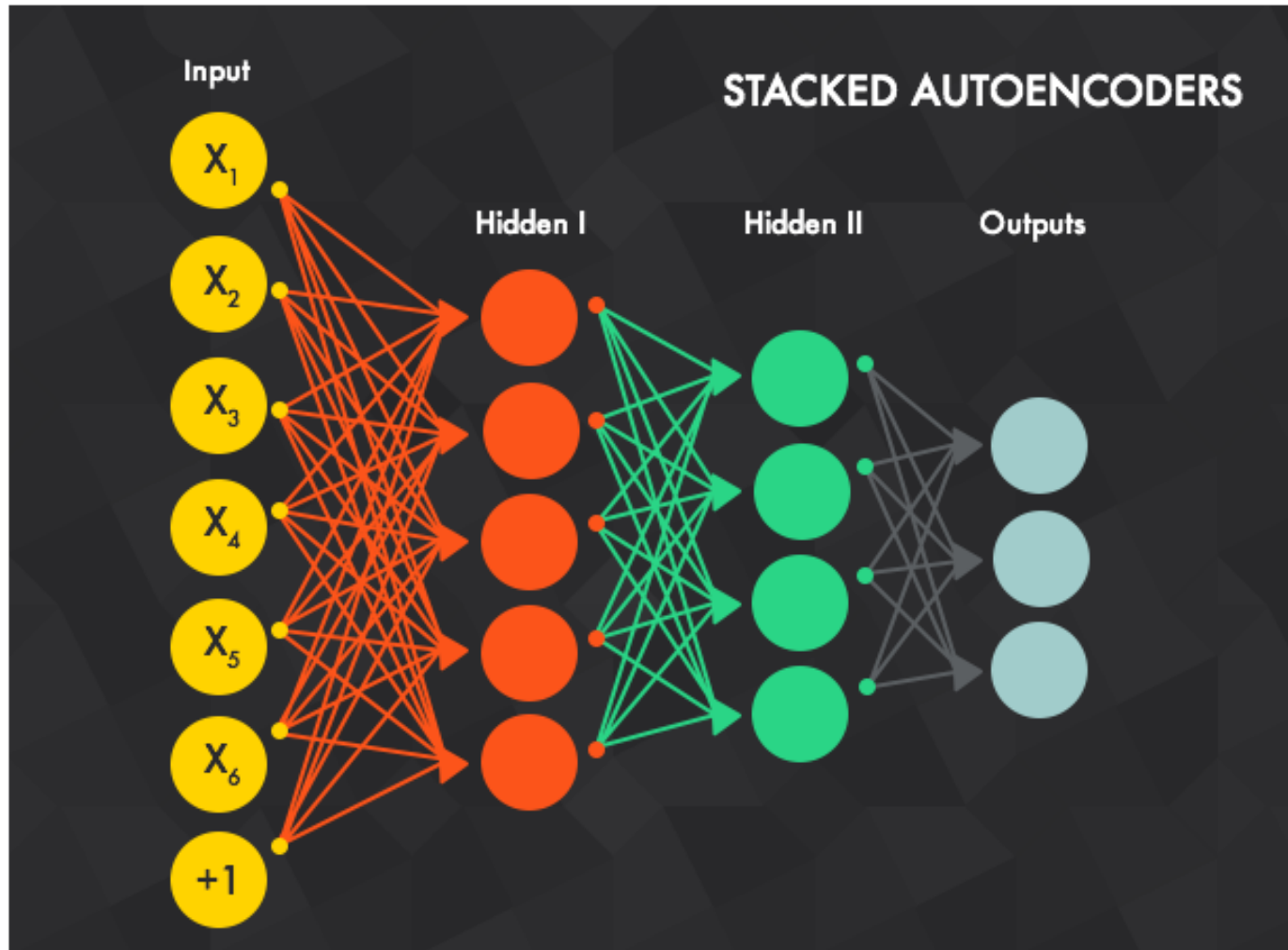
In so doing, we obtain a “feature extractor” realizing a **dimensionality reduction** of the original feature space.

Remark: this is an instance of the aforementioned 3rd kind of non-parametric estimator.

Let us move one step further:

1. let $\mathbb{R}^d$ be the feature space, and let $\tau = \{(\underline{x}, \underline{\hat{y}}) | \underline{x} \in \mathbb{R}^d, \underline{\hat{y}} \in \mathbb{R}^m\}$. Assume our goal is training a MLP (on τ) to realize $\varphi : \mathbb{R}^d \rightarrow \mathbb{R}^m$.
2. From τ we define $\tau' = \{(\underline{x}, \underline{x}) | (\underline{x}, \underline{\hat{y}}) \in \tau \text{ for some } \underline{\hat{y}}\}$ and we train the autoencoder on τ'
3. we remove the upper layer(s) from the autoencoder, and we map $\mathbb{R}^d$ onto $\mathbb{R}^{\hat{d}}$. From τ we obtain $\tau'' = \{(\underline{z}, \underline{\hat{y}}) | \underline{z} \in \mathbb{R}^{\hat{d}}\}$. In practice, we transform the original training patterns $\underline{x} \in \mathbb{R}^d$ into $\underline{z} \in \mathbb{R}^{\hat{d}}$ where $\hat{d} < d$, hence $\mathbb{R}^{\hat{d}}$ is a sub-space of $\mathbb{R}^d$.
4. Finally (as sought), a MLP is trained via backpropagation on τ'' , s.t. it realizes $\hat{\varphi} : \mathbb{R}^{\hat{d}} \rightarrow \mathbb{R}^m$
5. we mount the two MLPs on the top of each other, keeping their weights clamped, obtaining the overall function $\varphi : \mathbb{R}^d \rightarrow \mathbb{R}^m$ sought
6. later, the weights of the overall “deep” MLP (point 5) may be further tuned via backpropagation on τ , resulting in an improved model of φ

The procedure can be iterated obtaining **stacked autoencoders** built on the top of each other, progressively reducing the dimensionality of the feature space. This is a popular way to initialize and train deep ANNs



ANNs: pattern recognition and probability estimation

There are two ways of using the MLP as a non-parametric estimator for pattern recognition:

1. use MLPs as discriminant functions: as we said, train the MLP (like linear discriminants with Widrow-Hoff) via BP on a training set labeled with 0/1 target outputs, and assume the i -th output unit represents $g_i(\underline{x})$
2. probabilistic interpretation of the MLP output(s): we train one (or, more) MLP(s) for the (*non-parametric*) estimation of the probabilistic quantities involved in Bayes theorem: $(P(\omega_i|\underline{x}), \text{ or } p(\underline{x} | \omega_i))$.

The MLP output may be interpreted as a *probability* if and only if it is constrained within the $[0, 1]$ range. This is guaranteed if sigmoid activation functions are used.

If the MLP has c outputs and we want them to represent $P(\omega_1|\underline{x}), P(\omega_2|\underline{x}), \dots, P(\omega_c|\underline{x})$, respectively, the following additional constraint must be satisfied: $\sum_{i=1}^c P(\omega_i|\underline{x}) = 1$. This is granted if we let:

$$\hat{P}(\omega_i|\underline{x}) = \frac{y_i(\underline{x})}{\sum_{j=1}^c y_j(\underline{x})} \quad (9)$$

where $y_j(\underline{x})$ is the j -th ANN output over current input¹ $\underline{x}$.

Problem: what target output shall we use in order to train the MLP to model $P(\omega_i|\underline{x})$? The solution is yielded by the following

Theorem (Lippmann, Richard): by using the values 0 (if $\underline{x} \notin \omega_i$) or 1 (if $\underline{x} \in \omega_i$) as the target output for i -th unit, minimization of the criterion $C(w)$ (quadratic error) equals to minimizing the quadratic error between the i -th output and $P(\omega_i|\underline{x})$.

¹Use this notion also at training time.

In other words, if we reach the global minimum $C_{MIN}(\tilde{w})$ using 0/1 targets and the right MLP architecture, we are guaranteed that the MLP obtained this way (having weights $\tilde{w}$) is the optimal Bayesian classifier, as long as the class-posteriors are continuous.

In practice: using backpropagation on real-world data we never reach the global minimum, and the right MLP architecture is not known; therefore, the i -th output never coincides with $P(\omega_i|\underline{x})$. Moreover, in general $\sum_{i=1}^c y_i(\underline{x}) \neq 1$. This is why we need to impose the constraint $\sum_i \hat{P}(\omega_i|\underline{x}) = 1$ explicitly.

Another way of satisfying the constraint is to use a SOFTMAX normalization, by taking

$$\hat{P}(\omega_i|\underline{x}) \simeq y_i(\underline{x}) = \frac{e^{a_i}}{\sum_{j=1}^c e^{a_j}} \quad (10)$$

Q: may we use MLP as an estimate of the class-conditional pdfs? Since

$$y_i(\underline{x}) \simeq P(\omega_i|\underline{x}) = \frac{p(\underline{x}|\omega_i)P(\omega_i)}{p(\underline{x})} \quad (11)$$

we can write

$$\frac{y_i(\underline{x})}{P(\omega_i)} \simeq \frac{p(\underline{x}|\omega_i)}{p(\underline{x})} \quad (12)$$

which is known as the **scaled likelihood**.

If $p(\underline{x})$ were known, we could estimate the unknown pdf $p(\underline{x}|\omega_i)$ via MLP (provided that an estimate of $P(\omega_i)$ is used).

Although $p(\underline{x})$ is unknown under real-life circumstances, it can be estimated via statistical techniques (obtaining a hybrid neural-probabilistic machine) or via other ANNs. In fact, statistical estimates of $p(\underline{x})$ are more robust than individual estimates of $p(\underline{x}|\omega_i)$ for $i = 1, \dots, c$ (due to the fact that only one pdf has to be estimated instead of c , and the amount of data available for the estimation is c times larger on average).

Furthermore, should $P(\omega_i)$ change over test time (e.g., as a consequence of changed statistical conditions) assuming a new value $P'(\omega_i)$, we obtain the corresponding *new estimate* $P'(\omega_i|\underline{x})$ of the class-posterior as:

$$P'(\omega_i|\underline{x}) = \frac{p(\underline{x}|\omega_i)}{p(\underline{x})} P'(\omega_i) \simeq \frac{y_i(\underline{x})}{P(\omega_i)} P'(\omega_i) \quad (13)$$

which requires no re-training of the MLP.

Radial Basis Function (RBF) Networks

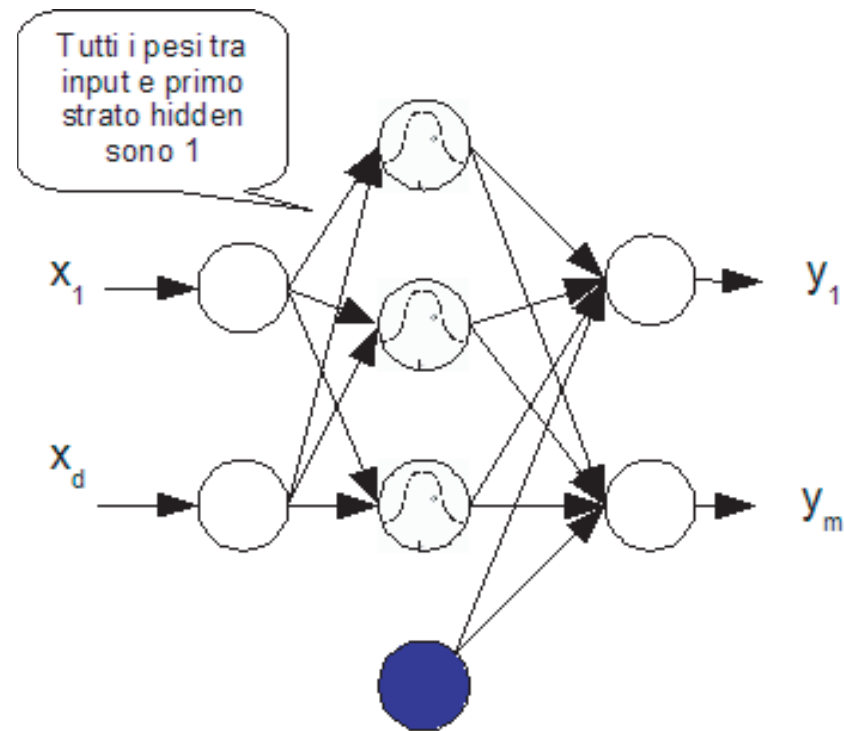


Figure: RBF

$$y_i = \sum_{j=1}^k w_{ij} \varphi_j(\underline{x}) + w_{io} \quad (14)$$

Note: it is a Generalized Linear Discriminant.

The RBF (or, kernel) is defined as $\varphi_k(\underline{x}) = e^{-\frac{\|\underline{x} - \underline{\mu}_k\|^2}{2\sigma_k^2}}$

Universality: just like MLPs, RBFs are “universal” approximators.

Learning (supervised): $C(\tau, w) = \frac{1}{2} \sum_i (\hat{y}_i - y_i)^2$.

Approach 1: via gradient descent over $C(w)$, learning the parameters w_{ij} , w_{io} , $\underline{\mu}_k$ and σ_k .

Approach 2: $\underline{\mu}_k$ and σ_k are **estimated statistically**, then the hidden-to-output weights (i.e., coefficients of the output linear transformation) are estimated via (i) linear algebra (say, matrix inversion) techniques, or (ii) gradient descent.

Remark: RBFs realize mixtures of Gaussian pdfs. Thence, they are particularly suitable to pdf estimation.

Maximum-likelihood (ML) criterion: we can apply *gradient-ascent over ML to the RBF in order to estimate pdfs*. As long as the hidden-to-output weights sum to one, we are good.

This does not work with MLPs, since the constraint $\int p(x)dx = 1$ is violated (**divergence problem**), since they realize mixtures of activation functions that are not inherently pdfs.

The Big Q: can we train the MLP to estimate a pdf?